DỰ ÁN HỌC MÁY TỪ ĐẦU ĐẾN CUỐI

Chương 4 - Huấn luyện Mô hình (Training Models)

TS. Trần Thành Thắng
Đại học Đông Á
August 11, 2026

Gemini Notebook

Nội dung Chương trình

1. Hồi quy tuyến tính & Phương trình chuẩn tắc
2. Hạ Gradient (Gradient Descent)
3. Hồi quy đa thức, Đường cong học tập & Chính quy hóa
4. Hồi quy Logistic & Softmax

Gemini Notebook

Mục tiêu bài học (1/2)

- Hiểu rõ cách thức hoạt động bên trong của các mô hình học máy thay vì coi chúng là ”hộp đen”.
- Tìm hiểu và áp dụng các phương pháp giải Hồi quy tuyến tính (Phương trình chuẩn tắc, SVD, Gradient Descent).
- Phân biệt được các loại Gradient Descent (Batch, Stochastic, Mini-batch) và ảnh hưởng của Tốc độ học (Learning Rate).

Gemini Notebook

Mục tiêu bài học (2/2)

- Hiểu cách xử lý dữ liệu phi tuyến tính bằng Hồi quy đa thức.
- Phát hiện Quá khớp (Overfitting) thông qua Đường cong học tập và các kỹ thuật Chính quy hóa (Ridge, Lasso, Elastic Net, Early Stopping).
- Tìm hiểu Hồi quy Logistic cho phân loại nhị phân và Hồi quy Softmax cho đa lớp.

Gemini Notebook

1. Hồi quy tuyến tính & Phương trình chuẩn tắc

Hồi quy tuyến tính (Linear Regression)

Hồi quy tuyến tính là một trong những mô hình đơn giản nhất.

Nguyên lý hoạt động:

Mô hình sẽ đưa ra dự đoán bằng cách tính tổng trọng số của các đặc trưng đầu vào, cộng với một hằng số gọi là:

- Số hạng độ lệch (bias term) hoặc
- Hệ số chặn (intercept term)

Gemini Notebook

Hồi quy tuyến tính: Dạng toán học

Mô hình dự đoán hồi quy tuyến tính (Dạng toán học):

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \quad (1)$$

Dạng Vector (Rút gọn):

$$\hat{y} = h_\theta(\boldsymbol{x}) = \theta \cdot \boldsymbol{x} \quad (2)$$

Trong đó:

- **θ** là vector tham số mô hình
- **x** là vector đặc trưng của trường hợp.

Gemini Notebook

Hàm chi phí MSE (Mean Squared Error)

Để huấn luyện mô hình Hồi quy tuyến tính, chúng ta cần tìm giá trị cho vector tham số θ nhằm tối thiểu hóa sai số giữa giá trị dự đoán và giá trị thực tế. Thước đo phổ biến nhất là Hàm chi phí Lỗi Bình phương Trung bình (MSE).

Công thức MSE cho mô hình hồi quy tuyến tính:

$$\mathrm{MSE}(\mathbf{X}, h_\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(\theta^\top \mathbf{x}^{(i)} - y^{(i)} \right)^2 \tag{3}$$

Mục tiêu của quá trình huấn luyện là tìm θ để MSE đạt giá trị nhỏ nhất.

Gemini Notebook

🔑 Phương trình chuẩn tắc (Normal Equation)

Để tìm giá trị θ làm tối thiểu hóa hàm chi phí, có một nghiệm dạng đóng (closed-form solution) – một phương trình toán học cho ra kết quả trực tiếp. Nó được gọi là **Phương trình chuẩn tắc**.

Công thức Phương trình chuẩn tắc:

$$\hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \tag{4}$$

- $\hat{\theta}$ là giá trị θ làm giảm thiểu hàm chi phí.
- **y** là vector của các giá trị mục tiêu (từ $y^{(1)}$ đến $y^{(m)}$).

Gemini Notebook

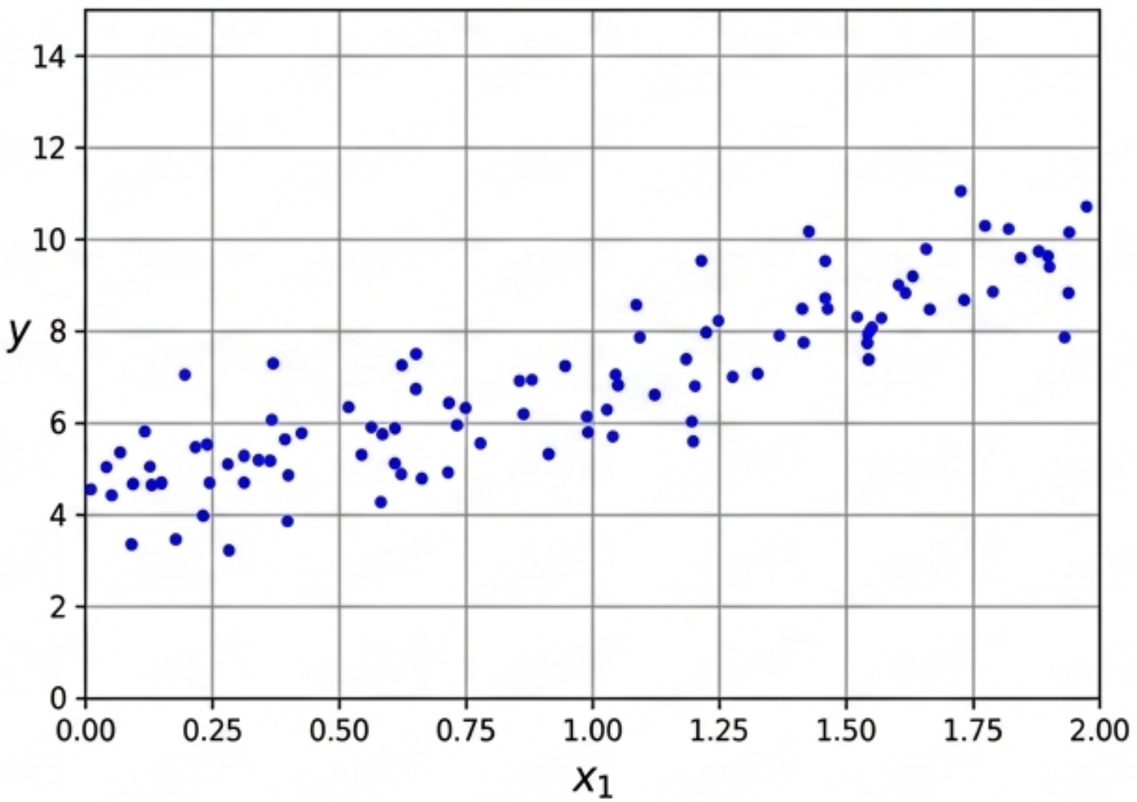
Minh họa: Tập dữ liệu tuyến tính ngẫu nhiên

Để kiểm tra Phương trình chuẩn tắc, chúng ta có thể tạo một số dữ liệu nhìn có vẻ tuyến tính một cách ngẫu nhiên.

Công thức sử dụng để tạo dữ liệu:

$$y = 4 + 3x + \text{nhiễu Gauss}$$

Biểu đồ bên cạnh cho thấy các điểm dữ liệu được tạo ra. Dữ liệu này sẽ được dùng để khớp mô hình.

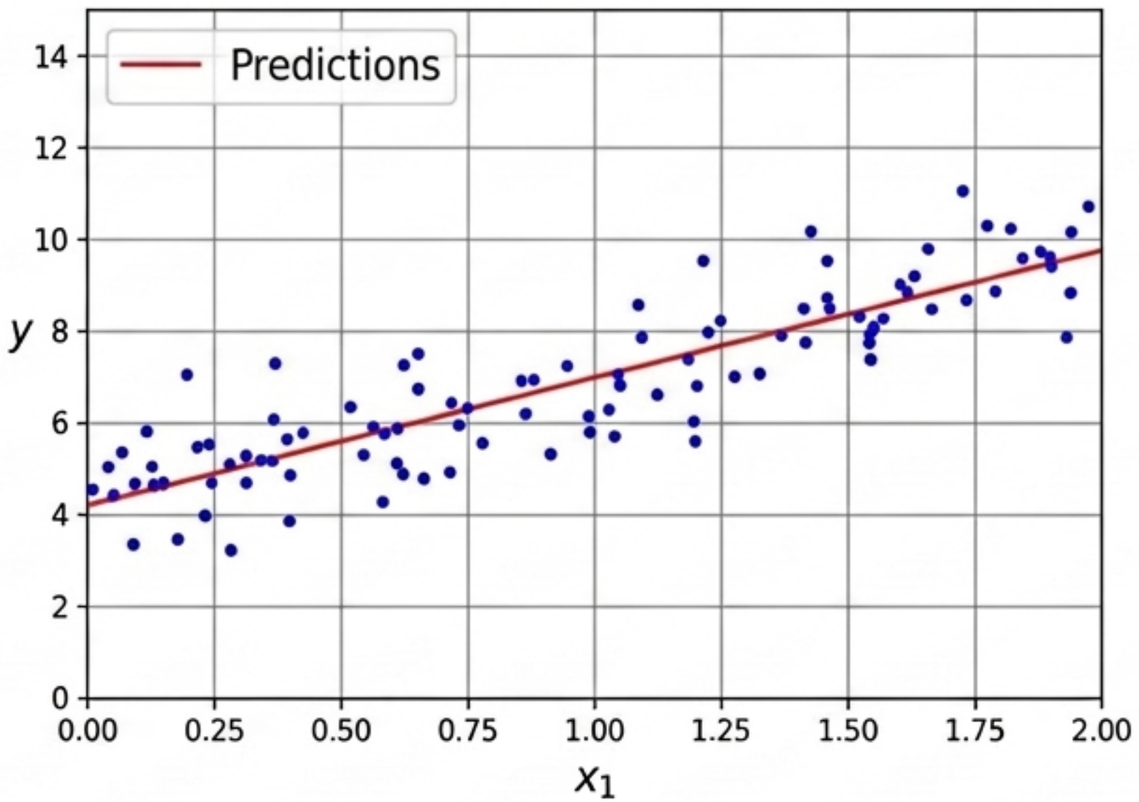


Hình 4-1. Dữ liệu tuyến tính được tạo ngẫu nhiên

Gemini Notebook

Minh họa: Dự đoán của Hồi quy tuyến tính

Sau khi tính toán được $\hat{\theta}$ từ Phương trình chuẩn tắc, chúng ta có thể vẽ đường thẳng dự đoán. Hình bên cạnh cho thấy các dự đoán của mô hình (đường thẳng màu đỏ) khớp với tập dữ liệu ngẫu nhiên của chúng ta như thế nào. Đường thẳng này đi qua trung tâm của các điểm dữ liệu, cho thấy mô hình đã học được xu hướng chung của dữ liệu.



Hình 4-2. Dự đoán của mô hình hồi quy tuyến tính

Gemini Notebook

Thực hiện với Scikit-Learn và SVD

Trong thực tế, Scikit-Learn sử dụng Singular Value Decomposition (SVD) thay vì tính trực tiếp Phương trình chuẩn tắc.

Lý do: Ma trận $\mathbf{X}^T\mathbf{X}$ có thể không khả nghịch (chẳng hạn khi số lượng đặc trưng lớn hơn số lượng trường hợp huấn luyện). Phương pháp SVD luôn xử lý được các trường hợp này bằng ma trận giả nghịch đảo (pseudoinverse).

```
lin_reg = LinearRegression()
lin_reg.fit(X, y)
```

Gemini Notebook

Độ phức tạp tính toán

Cả phương trình chuẩn tắc và cách tiếp cận SVD đều trở nên rất chậm khi số lượng đặc trưng tăng lên đáng kể.

Phương trình chuẩn tắc

- Việc đảo ngược ma trận có độ phức tạp khoảng $O(n^{2.4})$ đến $O(n^3)$.

Phương pháp SVD (Scikit-Learn)

- Độ phức tạp khoảng $O(n^2)$. (Tức là, nếu tăng gấp đôi số lượng đặc trưng, thời gian tính toán tăng gấp 4 lần).

Số lượng trường hợp huấn luyện

- Cả hai phương pháp đều là tuyến tính (độ phức tạp $O(m)$).

Ưu điểm: Chúng xử lý rất tốt tập huấn luyện lớn miễn là nó vừa với bộ nhớ.

Gemini Notebook

2. Hạ Gradient (Gradient Descent)

Gradient Descent là gì?

Gradient Descent (Hạ Gradient) là một thuật toán tối ưu hóa chung có khả năng tìm ra giải pháp tối ưu cho nhiều loại vấn đề.

Ý tưởng chung: Tinh chỉnh các tham số theo vòng lặp để giảm thiểu hàm chi phí.

Ví dụ trực quan:

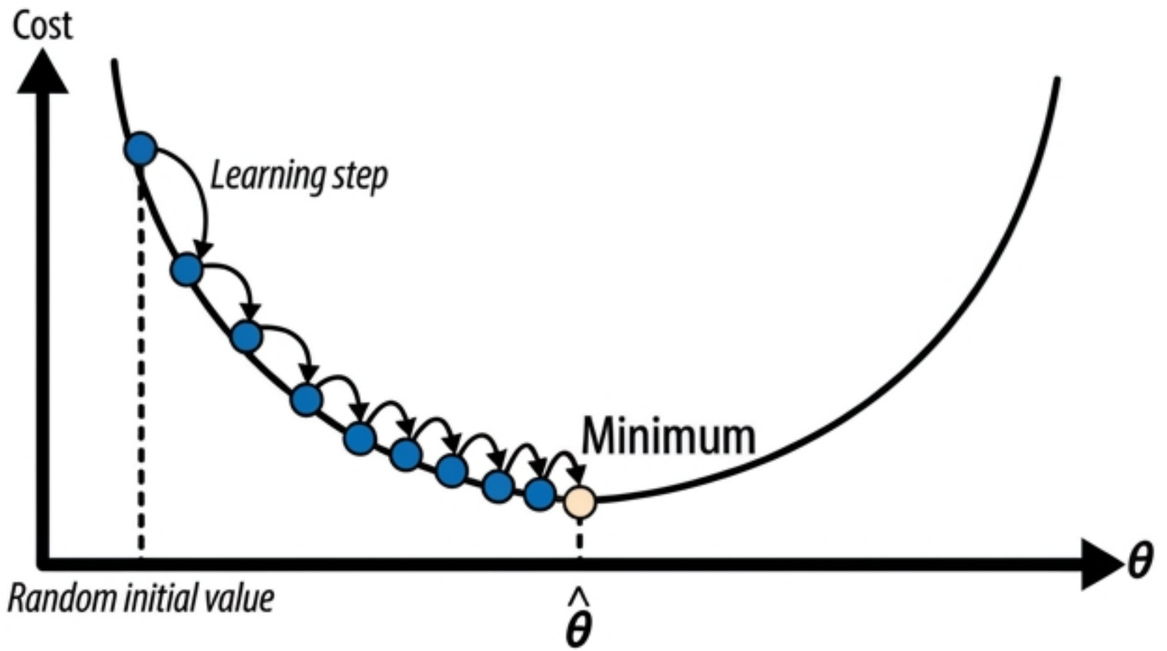
Tưởng tượng bạn bị lạc trên núi trong một màn sương mù dày đặc, và bạn chỉ có thể cảm nhận độ dốc của mặt đất dưới chân mình. Chiến lược tốt nhất để xuống đáy thung lũng là bước xuống dốc theo hướng dốc nhất.

Nó đo lường gradient (đạo hàm) cục bộ của hàm lỗi so với vector tham số **θ**, và đi theo hướng giảm của gradient cho đến khi gradient bằng 0.

Gemini Notebook

Nguyên lý hoạt động của Gradient Descent

- Kích thước bước nhảy được xác định bởi siêu tham số tốc độ học (learning rate).
- Bắt đầu với một giá trị θ ngẫu nhiên, thuật toán dần dần tiến gần đến điểm cực tiểu. Hình bên minh họa sự mô phỏng quá trình này: các bước đi dần hội tụ tại cực tiểu toàn cục.



Hình 4-3. Gradient Descent

Gemini Notebook

Phân tích ảnh hưởng của Tốc độ học (Learning Rate)

Tốc độ học (Learning Rate - η) là một siêu tham số quan trọng nhất của thuật toán Gradient Descent.

Tốc độ học quá nhỏ

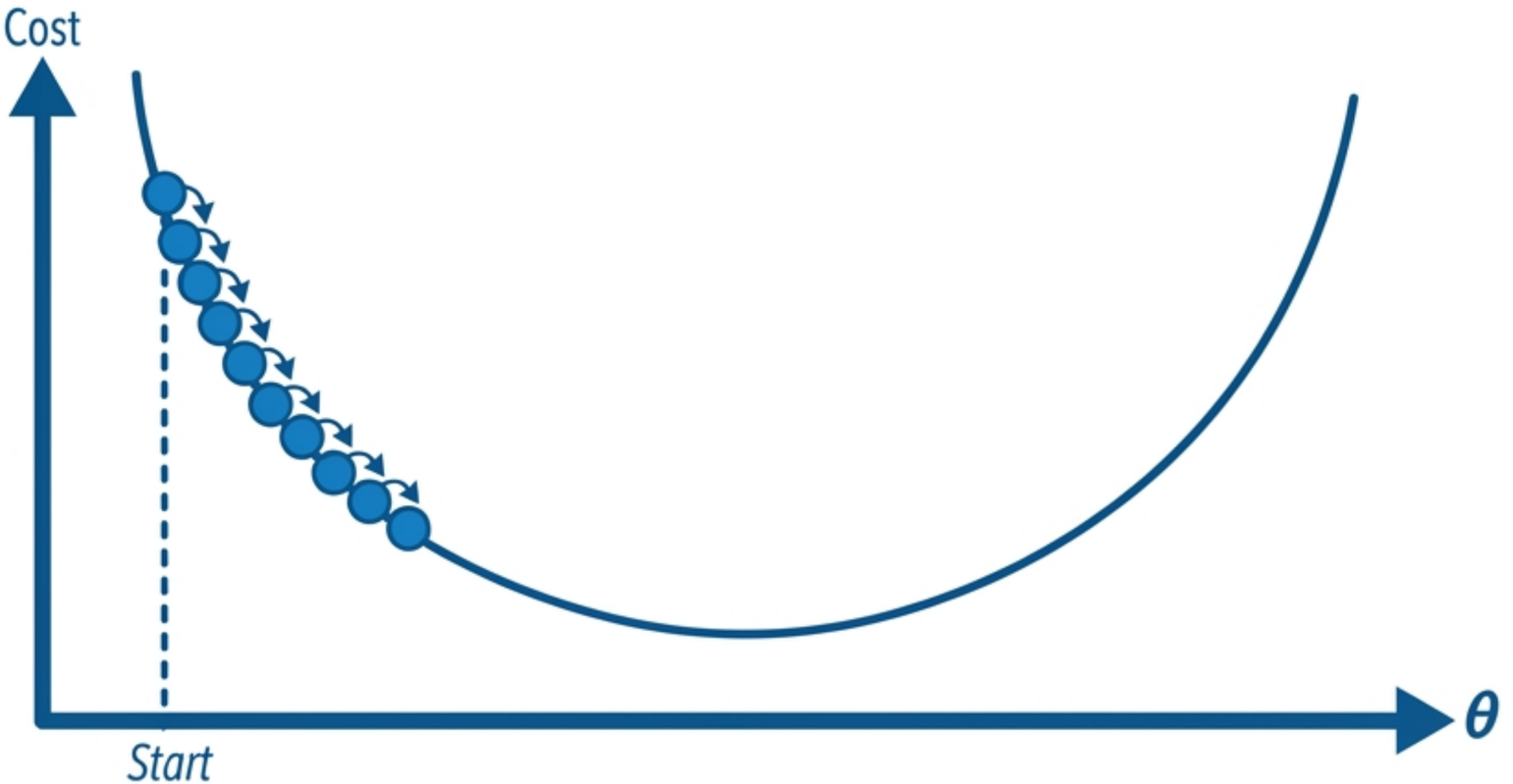
Thuật toán sẽ phải trải qua rất nhiều vòng lặp để hội tụ. Mất rất nhiều thời gian.

Tốc độ học quá cao

Bạn có thể "nhảy cóc" qua thung lũng và kết thúc ở bên kia, thậm chí cao hơn trước. Làm cho thuật toán phân kỳ (diverge), giá trị hàm chi phí ngày càng tăng và không bao giờ tìm được điểm tối ưu.

Gemini Notebook

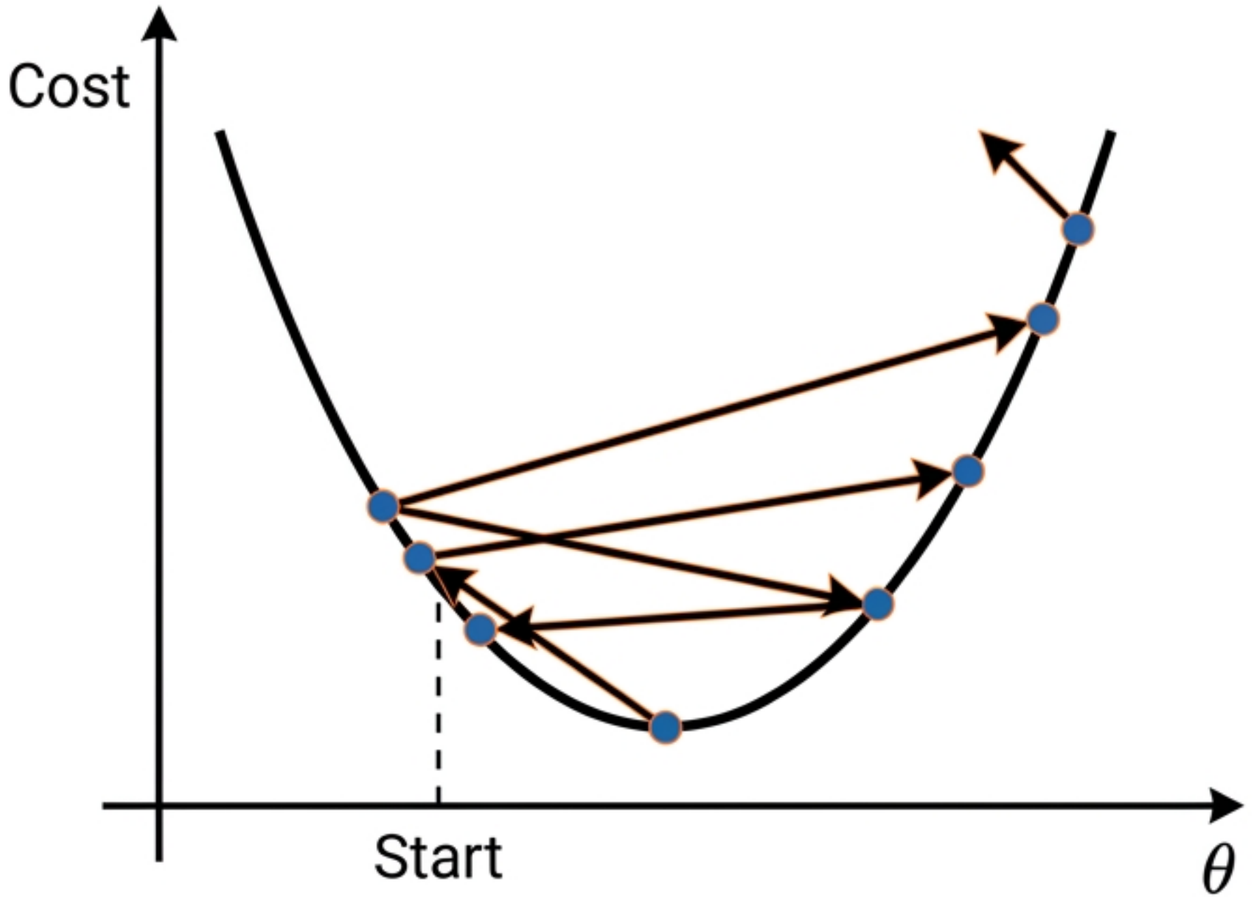
Minh họa: Tốc độ học quá nhỏ và quá cao



Hình 4-4. Tốc độ học quá nhỏ

Gemini Notebook

Minh họa: Tốc độ học quá nhỏ và quá cao



Hình 4-5. Tốc độ học quá lớn

Gemini Notebook

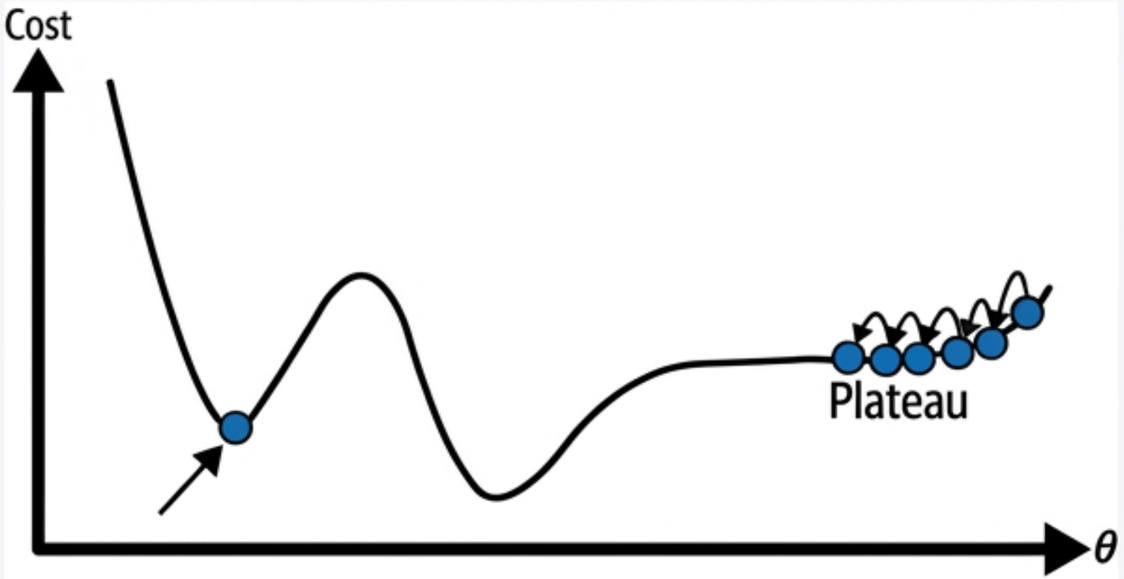
Các cạm bẫy của Gradient Descent

Không phải tất cả các hàm chi phí đều trông giống như những cái bát hình parabol hoàn hảo.

⚠️ **Cực tiểu cục bộ (Local Minimum):** Nếu bạn bắt đầu từ một điểm khởi tạo ngẫu nhiên, thuật toán có thể hội tụ về điểm thấp cục bộ chứ không phải cực tiểu toàn cục (Global Minimum).

Cao nguyên (Plateau): Hàm chi phí đi ngang, khiến thuật toán mất rất nhiều thời gian để đi qua và có thể bị dừng lại sớm.

✅ **Tin tốt:** Rất may, hàm chi phí MSE của hồi quy tuyến tính là hàm lồi (convex function).



Hình 4-6. *Cạm bẫy cực tiểu cục bộ*

Gemini Notebook

Tầm quan trọng của việc điều chỉnh đặc trưng (Scaling)

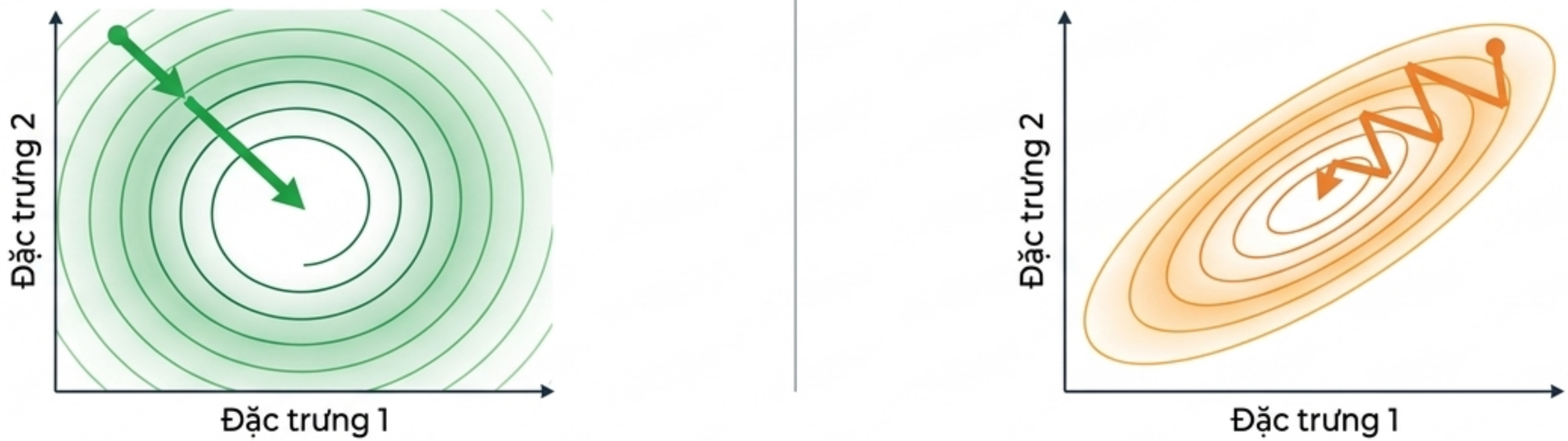
Khi sử dụng Gradient Descent, bạn phải đảm bảo rằng tất cả các đặc trưng có thang đo (scale) tương tự nhau (VD: dùng `StandardScaler`).

Bên trái

- Các đặc trưng được chia tỷ lệ bằng nhau.
- Hàm chi phí có dạng cái bát tròn, đường đi đến cực tiểu nhanh gọn và thẳng.

Bên phải

- Đặc trưng 1 nhỏ hơn Đặc trưng 2.
- Hàm chi phí có dạng bát kéo dài. Đường đi của thuật toán ngoằn ngoèo, tốn nhiều thời gian hội tụ.



Hình 4-7. Gradient Descent với và không có Scaling

Gemini Notebook

Hạ Gradient theo lô (Batch Gradient Descent)

the whole dataset

Batch Gradient Descent (Hạ Gradient theo lô) tính toán đạo hàm riêng của hàm chi phí đối với mỗi tham số θ_j bằng cách sử dụng **toàn bộ tập dữ liệu huấn luyện** tại mỗi bước lặp.

Vector Gradient của hàm chi phí:

$$\nabla_{\theta}\mathrm{MSE}(\theta) = \frac{2}{m} X^{\top}(X\theta - y) \quad (5)$$

Bước nhảy Gradient Descent:

$$\theta_{(\text{bước tiếp theo})} = \theta - \eta \nabla_{\theta}\mathrm{MSE}(\theta) \quad (6)$$

Mặc dù chậm trên tập dữ liệu rất lớn (do phải dùng toàn bộ dữ liệu), nhưng nó lại chia tỷ lệ **tốt** với số lượng đặc trưng lớn (tốt hơn Phương trình chuẩn tắc).

Gemini Notebook

Minh họa: Batch GD với các tốc độ học khác nhau

Hình bên cho thấy 10 bước đầu tiên của Gradient Descent theo lô.

Trái ($\eta = 0.02$)

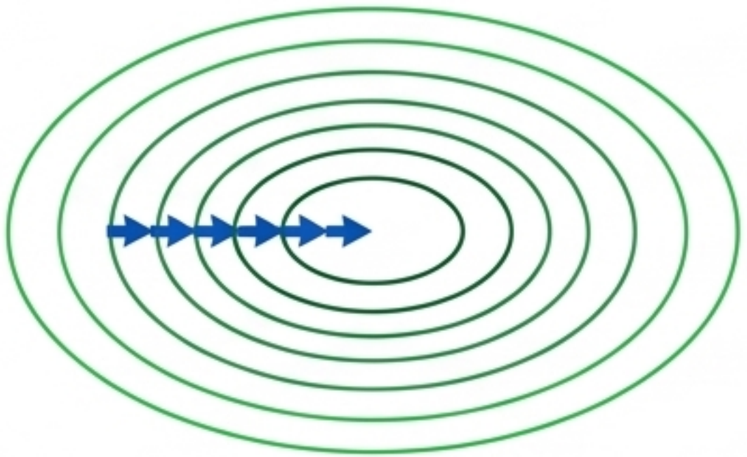
Tốc độ học quá thấp, sẽ đạt cực tiểu nhưng chậm.

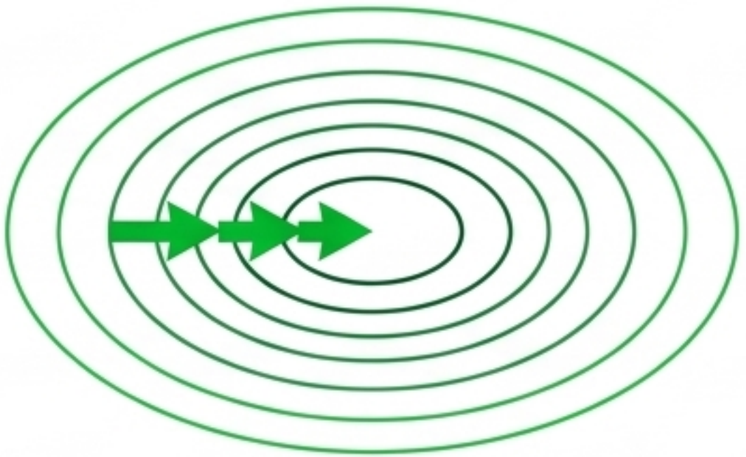
Giữa ($\eta = 0.1$)

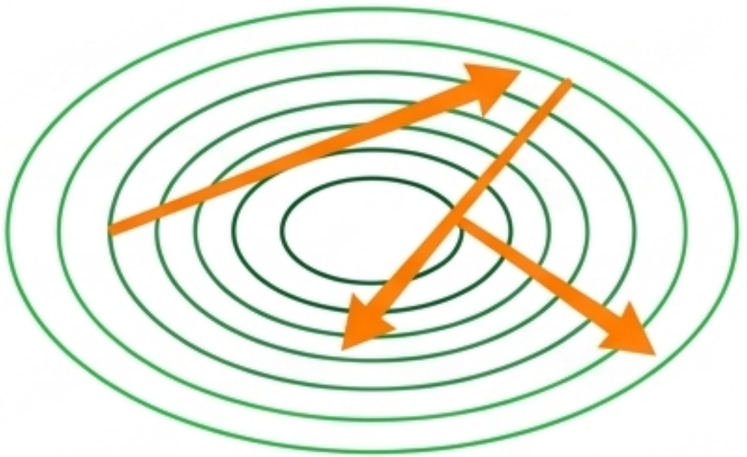
Tốc độ học rất tốt, thuật toán hội tụ nhanh chóng chỉ trong vài bước.

Phải ($\eta = 0.5$)

Tốc độ học quá cao, thuật toán nhảy loạn xạ và đang phân kỳ (diverging).







Hình 4-8. Batch GD với tốc độ học khác nhau

Gemini Notebook

Hạ Gradient ngẫu nhiên (Stochastic Gradient Descent - SGD)

Vấn đề chính của Batch GD là sử dụng toàn bộ tập dữ liệu huấn luyện để tính gradient tại mỗi bước, làm nó rất chậm với tập dữ liệu lớn.

Lựa chọn ngẫu nhiên:

SGD ngược lại, chọn ra một trường hợp ngẫu nhiên trong tập huấn luyện tại mỗi bước và chỉ tính gradient trên trường hợp duy nhất đó.

Tốc độ & Kích thước:

Điều này làm cho thuật toán cực kỳ nhanh, và có khả năng thực hiện huấn luyện các mô hình rất lớn vì mỗi lần lặp chỉ cần 1 trường hợp trong bộ nhớ (hỗ trợ Out-of-core learning).

Gemini Notebook

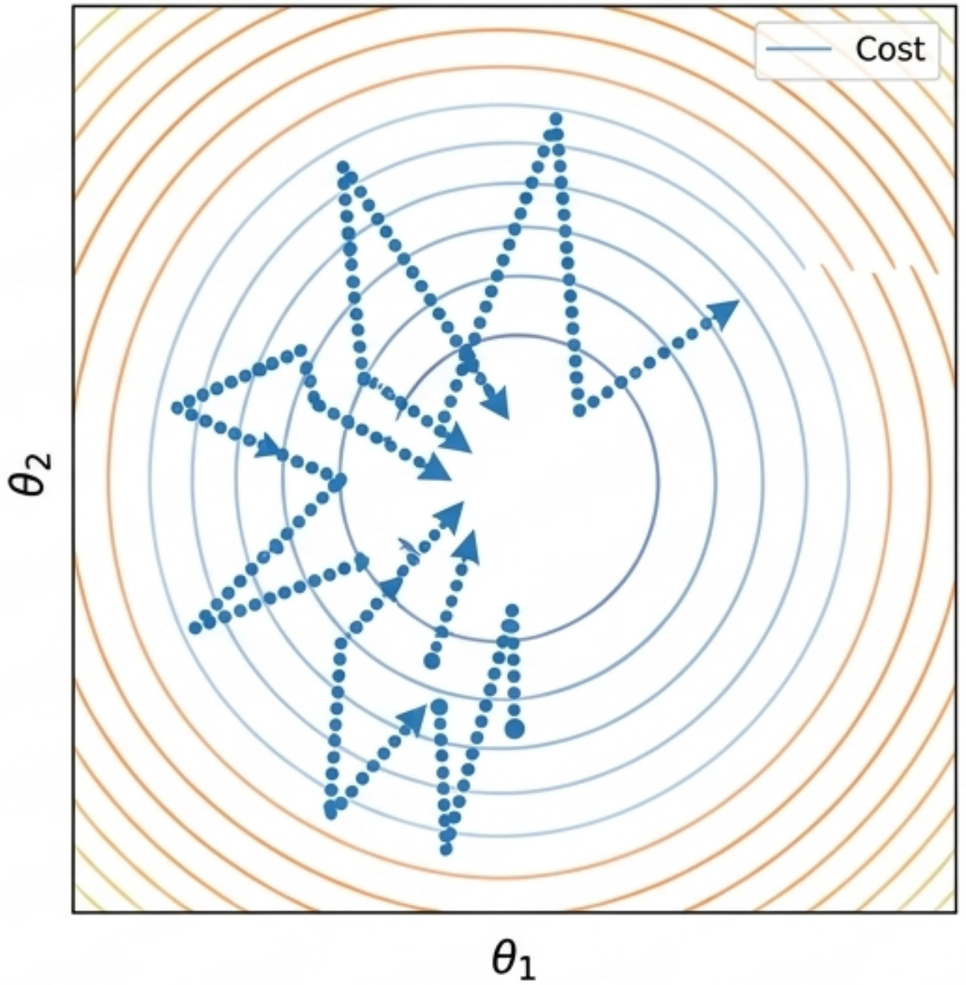
Đặc điểm đường đi của Stochastic Gradient Descent

Tính thất thường: Do tính chất ngẫu nhiên, SGD thất thường hơn nhiều so với Batch GD.

Dao động: Thay vì giảm nhẹ nhàng cho đến khi đạt mức tối thiểu, hàm chi phí sẽ dao động lên xuống, giảm dần về trung bình.

Kết quả: Ngay cả khi kết thúc gần cực tiểu, các giá trị tham số sẽ tiếp tục dao động. Nghĩa là giá trị cuối cùng là tốt, nhưng chưa phải tối ưu tuyệt đối.

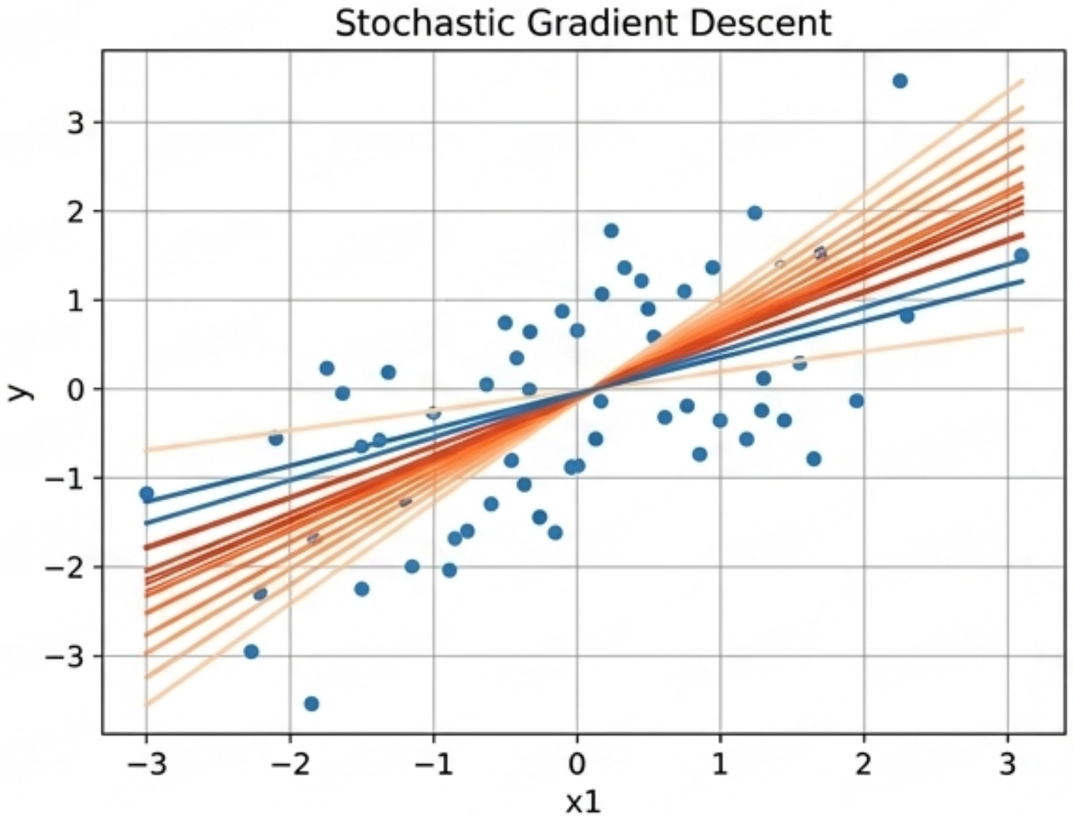
- **Ưu điểm:** Sự ngẫu nhiên giúp nó nhảy ra khỏi các cực tiểu cục bộ tốt hơn Batch GD.



Hình 4-9. Đặc điểm bất thường của SGD

Gemini Notebook

Minh họa: 20 bước đầu tiên của SGD



Hình 4-10. 20 bước đầu tiên của SGD

Hình bên cho thấy 20 bước đầu tiên của SGD. Bạn có thể thấy các bước không đều đặn như Batch GD.

Kỹ thuật lịch học (learning schedule): Để SGD hội tụ sát hơn, ta phải áp dụng kỹ thuật này: Tốc độ học lớn lúc đầu (tránh kẹt cực tiểu cục bộ), và giảm dần tốc độ học khi tiến về gần đích để ổn định tại đáy thung lũng.

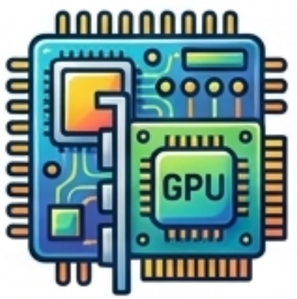
Gemini Notebook

Hạ Gradient theo Mini-Batch

Core Mechanism

Tập con ngẫu nhiên: Tại mỗi bước, thay vì tính toán gradient dựa trên toàn bộ dữ liệu (như Batch GD) hoặc chỉ một trường hợp (như SGD), thuật toán tính gradient trên các tập con nhỏ ngẫu nhiên gọi là **mini-batch**.

Advantages & Trade-offs

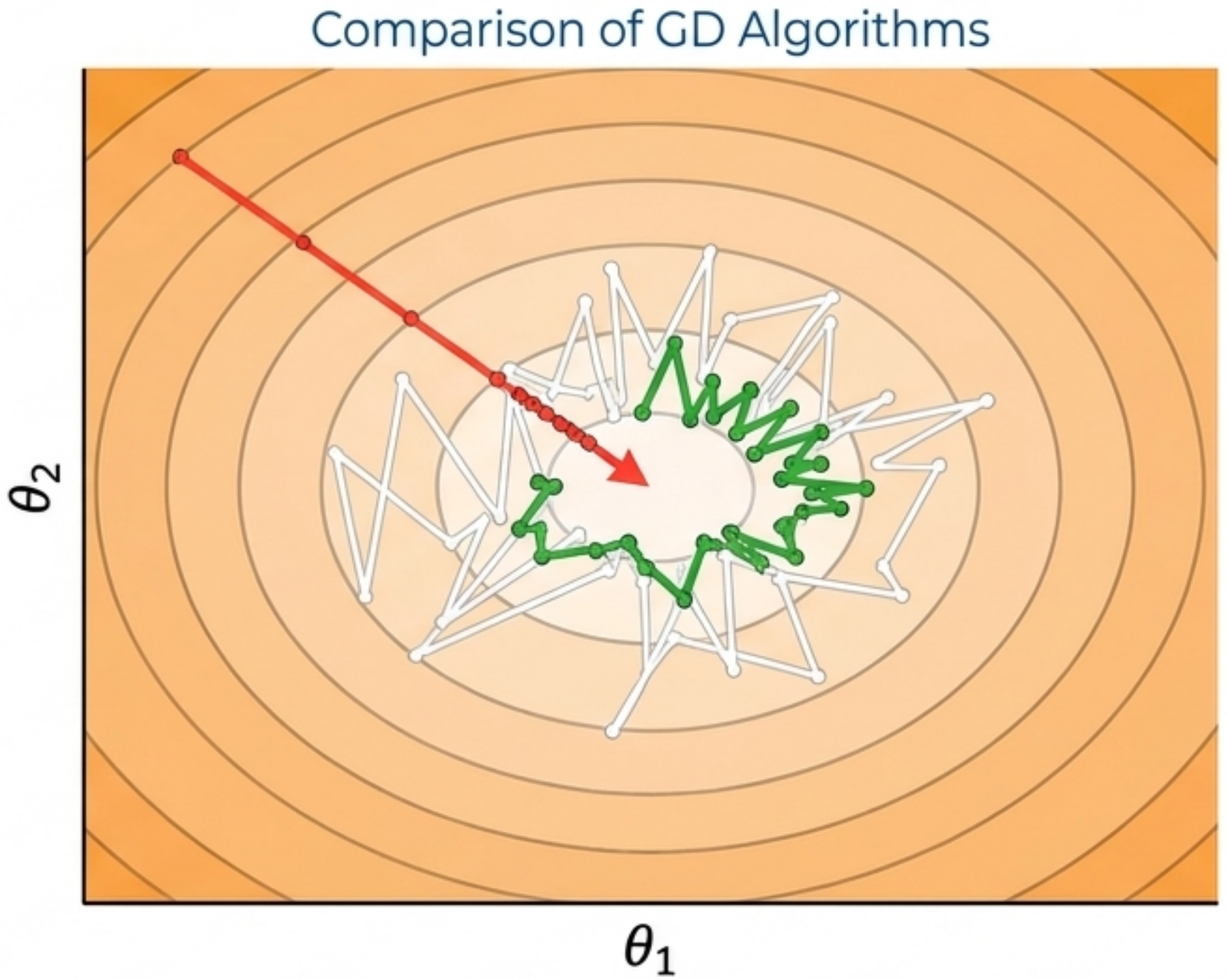


- **Tối ưu phần cứng:** Ưu điểm lớn nhất là bạn có thể tăng hiệu suất từ việc tối ưu hóa phần cứng của các phép toán ma trận, đặc biệt khi sử dụng GPUs.
- **Sự ổn định:** Sự tiến bộ của thuật toán ít thất thường hơn SGD (với mini-batch khá lớn).
- **Đích đến:** Nó sẽ tiến gần cực tiểu hơn SGD một chút, nhưng có thể khó thoát cực tiểu cục bộ hơn.

Gemini Notebook

So sánh đường đi của 3 thuật toán Gradient Descent

- **Batch GD** (Đỏ): Điểm đến thẳng cực tiểu và dừng lại, nhưng tốn nhiều thời gian mỗi bước.
- **SGD** (**Trắng**): Dao động mạnh xung quanh đích.
- **Mini-Batch GD** (**Xanh**): Ít dao động hơn SGD và tiến gần đích hơn, nhưng vẫn không dừng lại **chính xác** tại một điểm trừ khi có lịch học (learning schedule) tốt.



Hình 4-11. *Đường đi của 3 thuật toán GD*

Gemini Notebook

Bảng so sánh tổng hợp các thuật toán hồi quy tuyến tính

Thuật toán	Lớn m (trường hợp)	Ngoài lõi	Lớn n (đặc trưng)	Hyperparams	Scaling	Scikit-Learn
Phương trình chuẩn tắc	Nhanh	Không	Chậm	0	Không	N/A
SVD	Nhanh	Không	Chậm	0	Không	LinearRegression
Batch GD	Chậm	Không	Nhanh	2	Có	N/A
Stochastic GD	Nhanh	Có	Nhanh	>=2	Có	SGDRegressor
Mini-batch GD	Nhanh	Có	Nhanh	>=2	Có	N/A

Lưu ý: Sau khi huấn luyện, tất cả các thuật toán này đều tạo ra mô hình rất giống nhau và đưa ra dự đoán theo cùng một cách.

Gemini Notebook

The Clean Math Lab

3. Hồi quy đa thức, Đường cong học tập & Chính quy hóa

Hồi quy đa thức (Polynomial Regression)

Điều gì sẽ xảy ra nếu dữ liệu của bạn phức tạp hơn một đường thẳng? Thật đáng ngạc nhiên, bạn có thể sử dụng một mô hình tuyến tính để khớp với dữ liệu phi tuyến tính.

Gemini Notebook

Hồi quy đa thức (Polynomial Regression)

Core Concept: Cách đơn giản: Thêm các lũy thừa của mỗi đặc trưng làm đặc trưng mới, sau đó huấn luyện mô hình tuyến tính trên tập đặc trưng mở rộng này. Kỹ thuật này được gọi là **Hồi quy đa thức**.

Implementation Details:

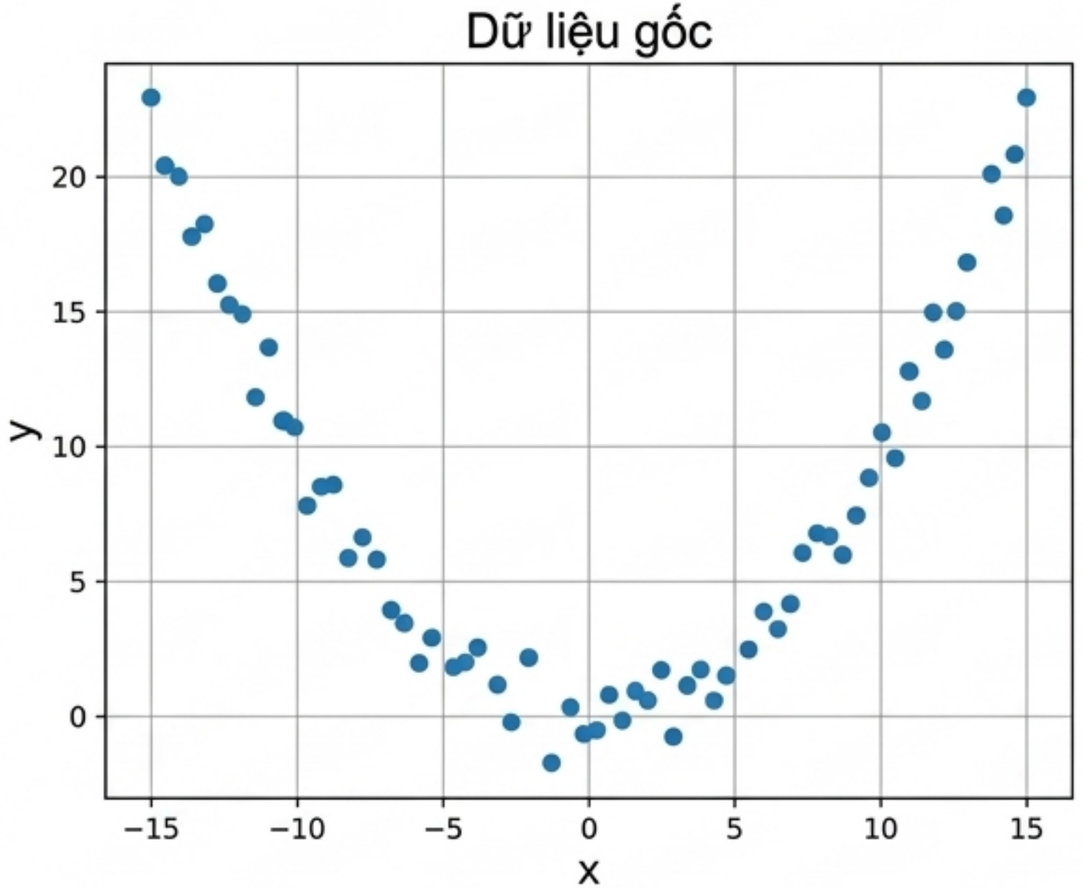
- `PolynomialFeatures` trong Scikit-Learn có thể dễ dàng tạo ra các biến bậc cao và kết hợp biến (tương tác giữa các đặc trưng).

Ví dụ thực tế: Giả sử chúng ta có một tập dữ liệu phân bố theo phương trình bậc hai ($y = ax^2 + bx + c$ + nhiễu). Một đường thẳng tuyến tính bình thường không bao giờ khớp được (Underfitting). Bằng cách dùng PolynomialFeatures(degree=2), chúng ta thêm x^2 vào bộ đặc trưng. Sau đó dùng LinearRegression khớp dữ liệu mới.

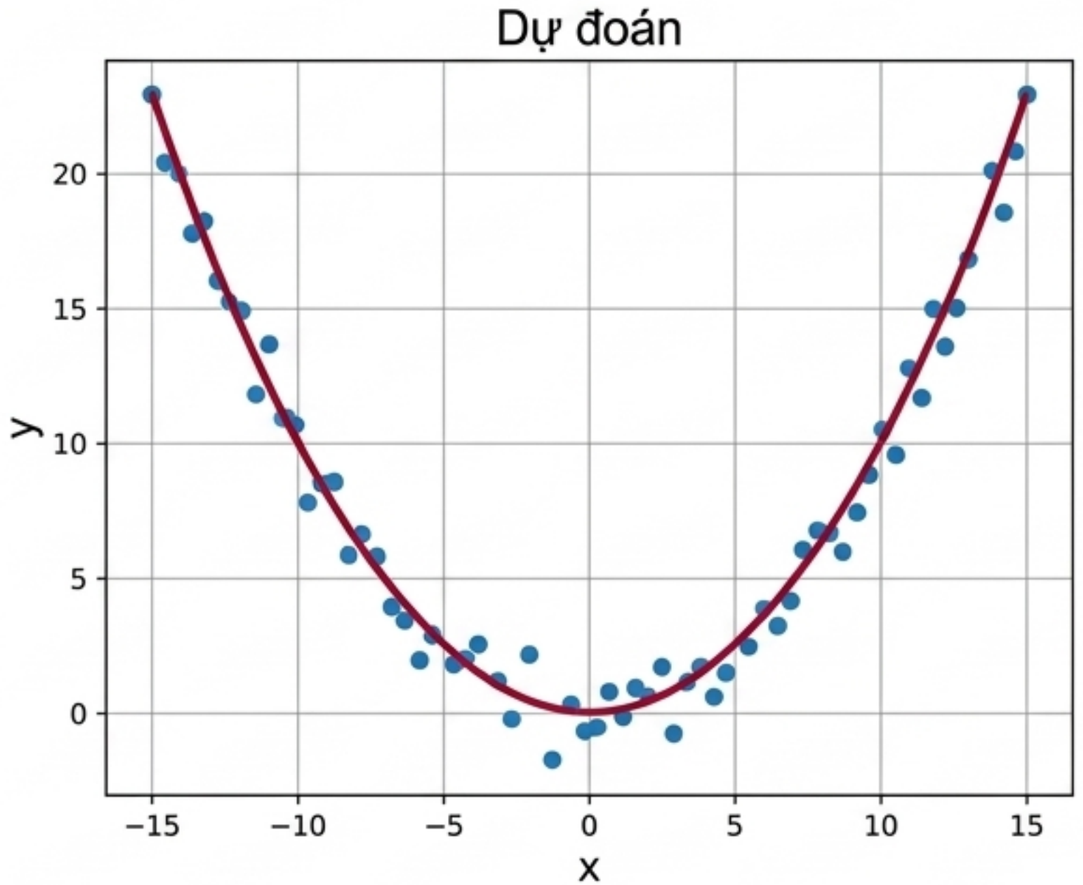
Gemini Notebook

Minh họa: Khớp Hồi quy đa thức trên dữ liệu phi tuyến

Kết quả thu được là một đường parabol bám rất sát dữ liệu (như Hình 4-13).



Hình 4-12. Tập dữ liệu phi tuyến tính ngẫu nhiên



Hình 4-13. Dự đoán mô hình hồi quy đa thức

Gemini Notebook

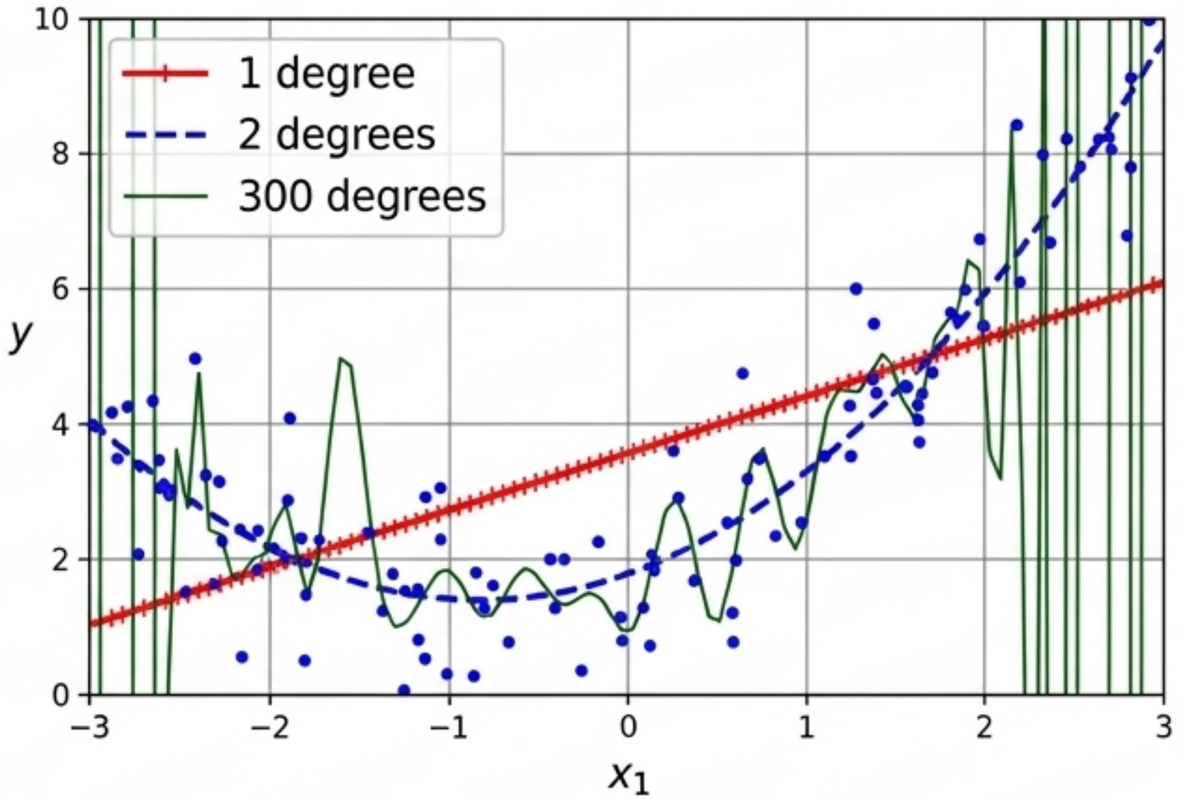
Machine Learning: Đánh Giá Mô Hình & Chính Quy Hóa

Chương 4 - Bài giảng giới thiệu bậc đại học

Gemini Notebook

Hiện tượng Quá khớp (Overfitting) với đa thức bậc cao

- Thực hiện hồi quy đa thức bậc quá cao (Vd: bậc 300) khiến mô hình khớp dữ liệu huấn luyện cực kỳ tốt (đường uốn lượn dữ dội).
- Đây là **Quá khớp nghiêm trọng**. Mô hình bậc 2 (Quadratic) mới là mô hình **tổng quát hóa** (Generalize) tốt nhất.
- Câu hỏi đặt ra: Làm sao nhận biết mô hình đang quá khớp hay dưới khớp? -> Sử dụng **Đường cong học tập**.



Hình 4-14. Hồi quy đa thức bậc cao (Overfitting)

Gemini Notebook

Đánh giá mô hình bằng Đường cong học tập (Learning Curves)

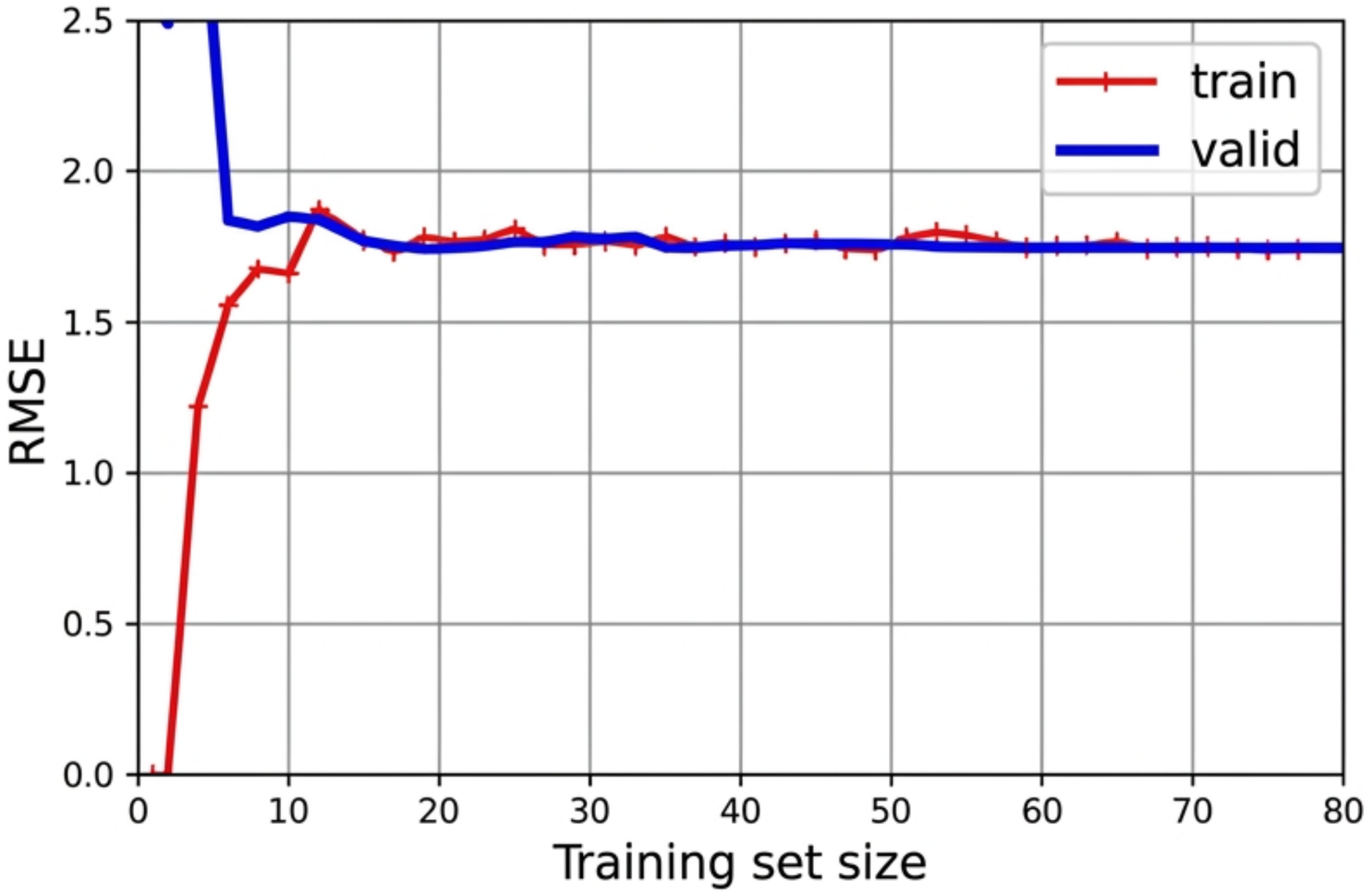
Đường cong học tập thể hiện lỗi trên tập Huấn luyện (Train) và tập Xác thực (Validation) dưới dạng hàm của kích thước tập huấn luyện.

Dưới khớp (Underfitting): Cả hai đường đều đạt cao nguyên, lỗi cao và sát nhau. Việc thêm dữ liệu không mang lại sự cải thiện.

Quá khớp (Overfitting): Lỗi trên tập huấn luyện thấp hơn nhiều. Xuất hiện khoảng cách (gap) đáng kể giữa hai đường (Mô hình tốt trên tập huấn luyện nhưng kém trên dữ liệu mới).

Gemini Notebook

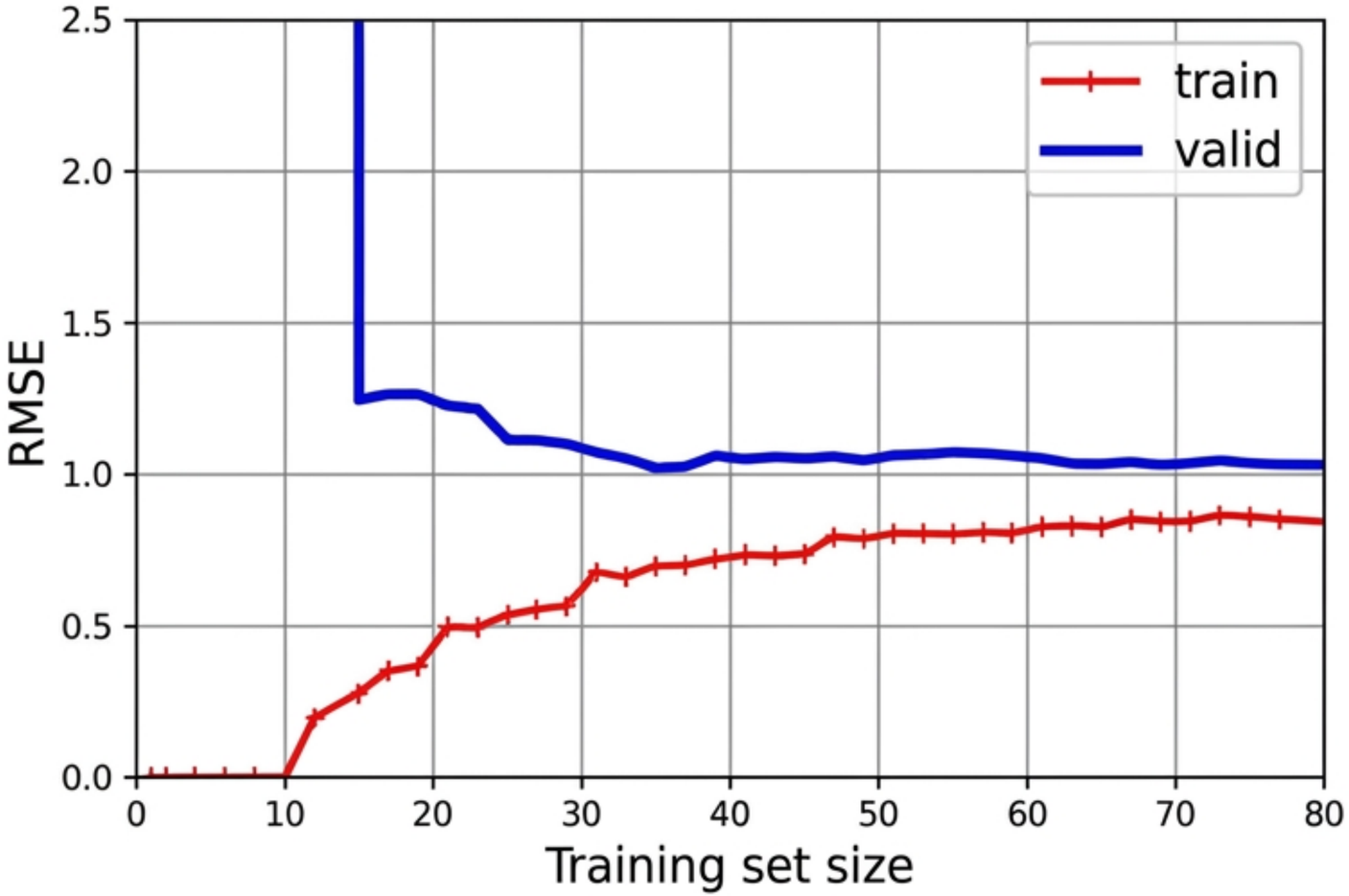
Minh họa: Đường cong học tập (Dưới khớp)



Hình 4-15. Dưới khớp (Đường cong sát nhau, lỗi cao)

Gemini Notebook

Minh họa: Đường cong học tập (Quá khớp)



Hình 4-16. Quá khớp (Khoảng cách rõ ràng giữa 2 đường cong)

Gemini Notebook

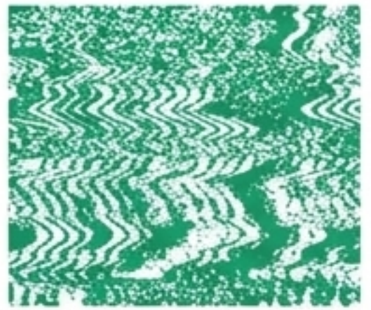
Sự đánh đổi giữa Độ chệch (Bias) và Phương sai (Variance)

Độ chệch (Bias)

Lỗi do các giả định sai (Vd: giả định tuyến tính cho dữ liệu phi tuyến). Bias cao gây ra underfitting.

Phương sai (Variance)

Lỗi do mô hình quá nhạy cảm với biến động nhỏ. Bậc tự do cao -> variance cao -> overfitting.



Lỗi không thể giảm (Irreducible error)

Độ nhiễu của bản thân dữ liệu. Chỉ giảm được bằng cách làm sạch dữ liệu.

Sự đánh đổi: Tăng độ phức tạp → Tăng variance, giảm bias.
Giảm độ phức tạp → Tăng bias, giảm variance.

Gemini Notebook

Chính quy hóa (Regularization): Hồi quy Ridge

Giảm **Overfitting** bằng cách chính quy hóa (ràng buộc) kích thước của các trọng số **θ**.

Hồi quy **Ridge** (Chuẩn hóa Tikhonov) thêm số hạng phạt **l2** vào hàm chi phí MSE:

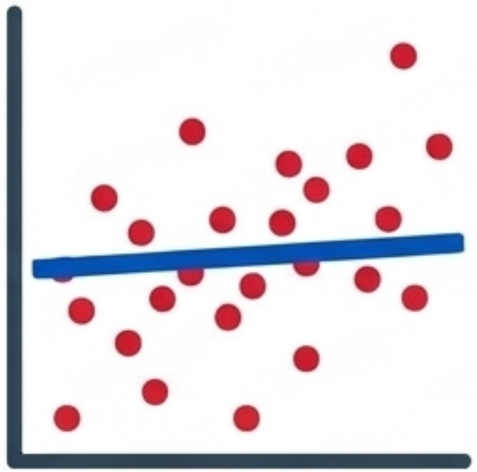
$$J(\theta) = \mathrm{MSE}(\theta) + \alpha \frac{1}{2} \sum_{i=1}^{n} \theta_i^2 \qquad (7)$$

α kiểm soát mức độ chính quy hóa.

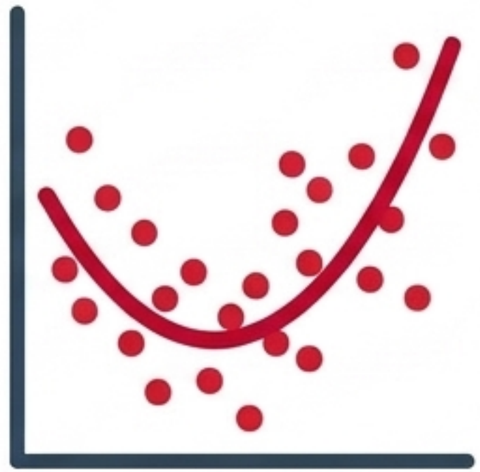
- **α = 0**: Hồi quy tuyến tính bình thường.
- **α lớn**: Trọng số **θ** gần về 0, tạo đường thẳng phẳng cắt qua trung bình dữ liệu.

Gemini Notebook

Minh họa: Hồi quy Ridge



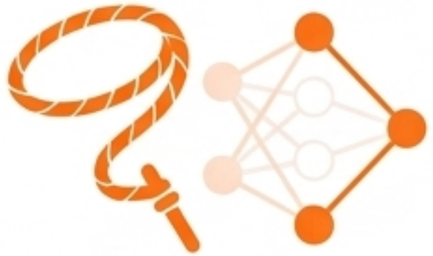
Mô hình Tuyến tính: Áp dụng trên mô hình tuyến tính thông thường. Khi α tăng, đường dự đoán trở nên phẳng hơn, giúp giảm độ chệch của các giá trị cực đoan.



Mô hình Đa thức: Áp dụng sau biến đổi PolynomialFeatures(degree=10). Nếu α = 0, đường cong dao động cực mạnh. Khi α lớn, đường cong êm hơn nhiều, giảm overfitting đáng kể.

Hình 4-17. Hồi quy Ridge trên mô hình tuyến tính và đa thức

Gemini Notebook



Chính quy hóa (Regularization): Hồi quy Lasso

Hồi quy Lasso (Least Absolute Shrinkage and Selection Operator) Operator) sử dụng chuẩn l1 làm số hạng phạt:

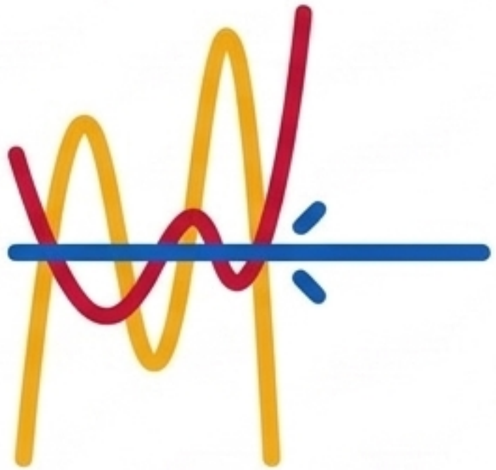
$$J(\theta) = \mathrm{MSE}(\theta) + \alpha \sum_{i=1}^{n} |\theta_i| \tag{8}$$

Điểm đặc biệt nhất: Lasso có xu hướng **hoàn toàn loại bỏ** (đặt trọng số = 0) các đặc trưng ít quan trọng. Kết quả: **Tự động lựa chọn** đặc trưng (feature selection), tạo ra một **mô hình thưa** (sparse model).

Gemini Notebook

Minh họa: Hồi quy Lasso

Dữ liệu tuyến tính: Lasso áp dụng hình phạt **chuẩn l1**, dẫn đến việc **giảm mạnh** các hệ số không quan trọng. **loại bỏ**.



Dữ liệu đa thức: Điểm đáng chú ý: Với $\alpha = 1$, đường dự đoán đa thức gần như trở thành một **đường thẳng bậc thấp**. Điều này chứng tỏ các tham số bậc cao đã bị đưa về **giá trị 0** hoàn toàn.

Hình 4-18. Hồi quy Lasso trên mô hình tuyến tính và đa thức

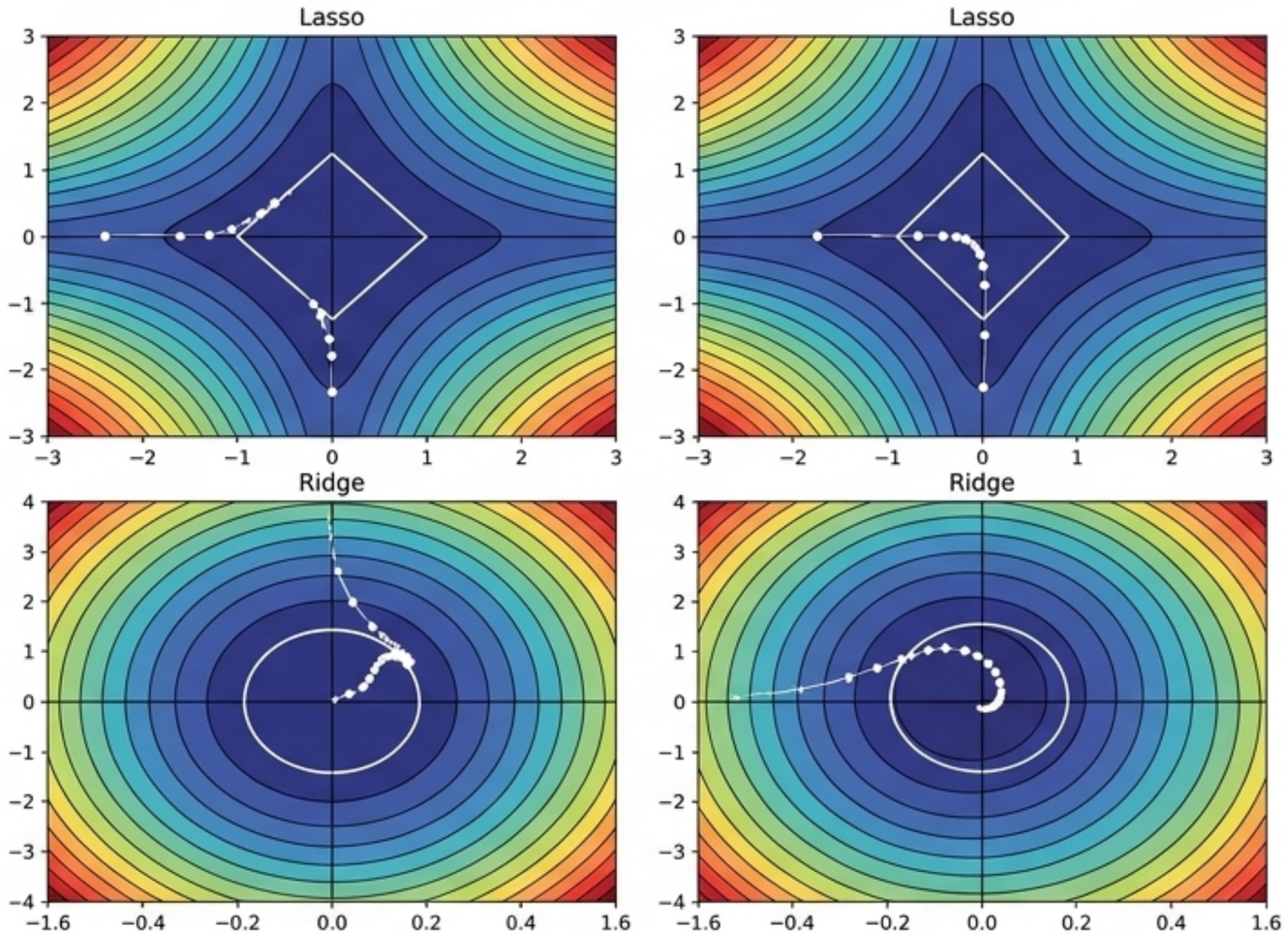
Gemini Notebook

Điểm khác biệt giữa Lasso và Ridge trong không gian tham số

Ridge (Chuẩn l2)	**Lasso (Chuẩn l1)**
• **Trọng số giảm dần** khi tiến tới gốc tọa độ. • **Gradient nhỏ dần** -> Tối ưu **hội tụ êm**, không dao động. • Các tham số **không bao giờ bị loại bỏ hoàn toàn** (không về 0).	• **Hàm chi phí giảm tuyến tính** khi tiến tới các trục (θ1, θ2). • **Hạ gradient** khiến một tham số nhanh chóng **về 0**, sau đó trượt trên trục còn lại. • Kết quả: Tạo ra **mô hình thưa, loại bỏ đặc trưng**.

Gemini Notebook

Minh họa: Lasso vs Ridge trong Không gian tham số



Hình 4-19. Chính quy hóa Lasso (Trên) và Ridge (Dưới)

Gemini Notebook

Hồi quy **Elastic Net** & Khi nào nên dùng?

Elastic Net kết hợp cả chuẩn l1 và l2 (tỷ lệ trộn r). r = 0 -> Ridge; r = 1 -> Lasso.

Nguyên tắc lựa chọn (Luôn ưu tiên dùng một ít chính quy hóa):

- ✅ **Ridge**: Lựa chọn mặc định tốt nhất.
- ✅ **Lasso** hoặc **Elastic Net**: Dùng khi nghi ngờ chỉ có vài đặc trưng là hữu ích (giúp loại bỏ nhiễu).
- ✅ **Elastic Net > Lasso**: Elastic Net được ưa chuộng hơn vì Lasso có thể hoạt động thất thường khi số lượng đặc trưng lớn hơn số trường hợp, hoặc các đặc trưng có tương quan mạnh.

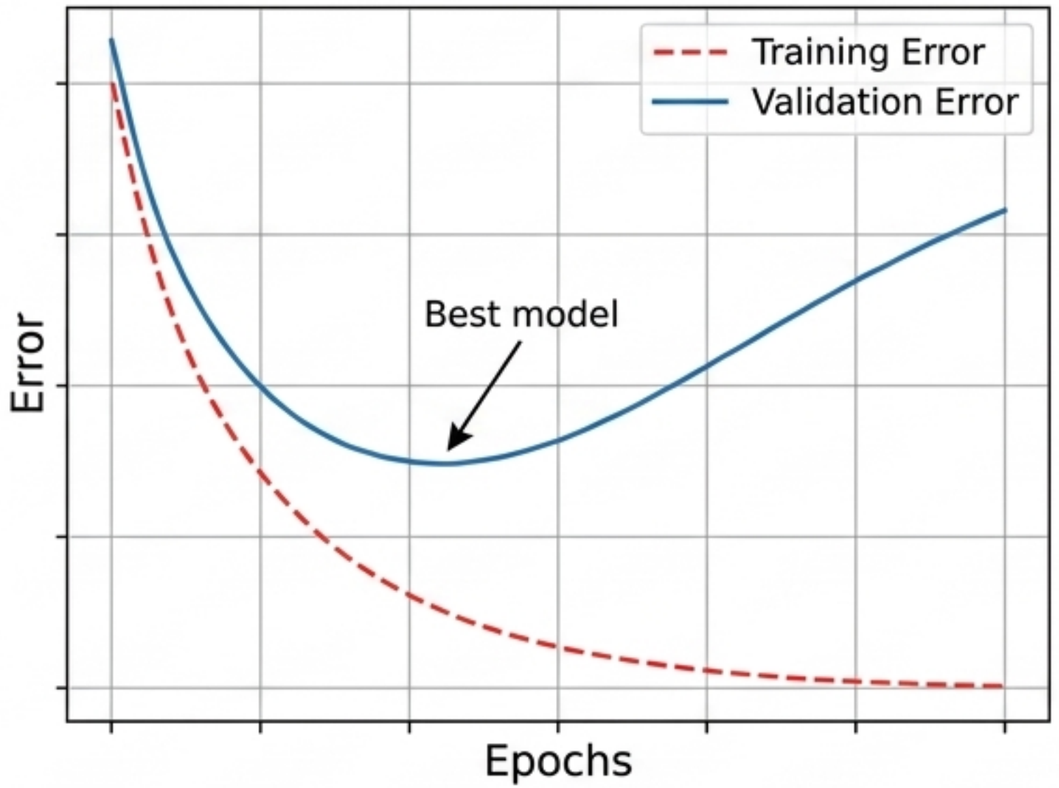
Gemini Notebook

Dừng sớm (Early Stopping)

Dừng huấn luyện thuật toán học lặp (như Gradient Descent) ngay khi Lỗi xác thực (Validation Error) đạt đến cực tiểu.

Lỗi huấn luyện luôn giảm. Lỗi xác thực ban đầu giảm, nhưng sau đó tăng ngược trở lại (bắt đầu Overfitting). Điểm chạm đáy chính là mô hình tốt nhất.

🎁 Kỹ thuật này hiệu quả và đơn giản đến mức Geoffrey Hinton gọi nó là "bữa trưa miễn phí tuyệt vời".



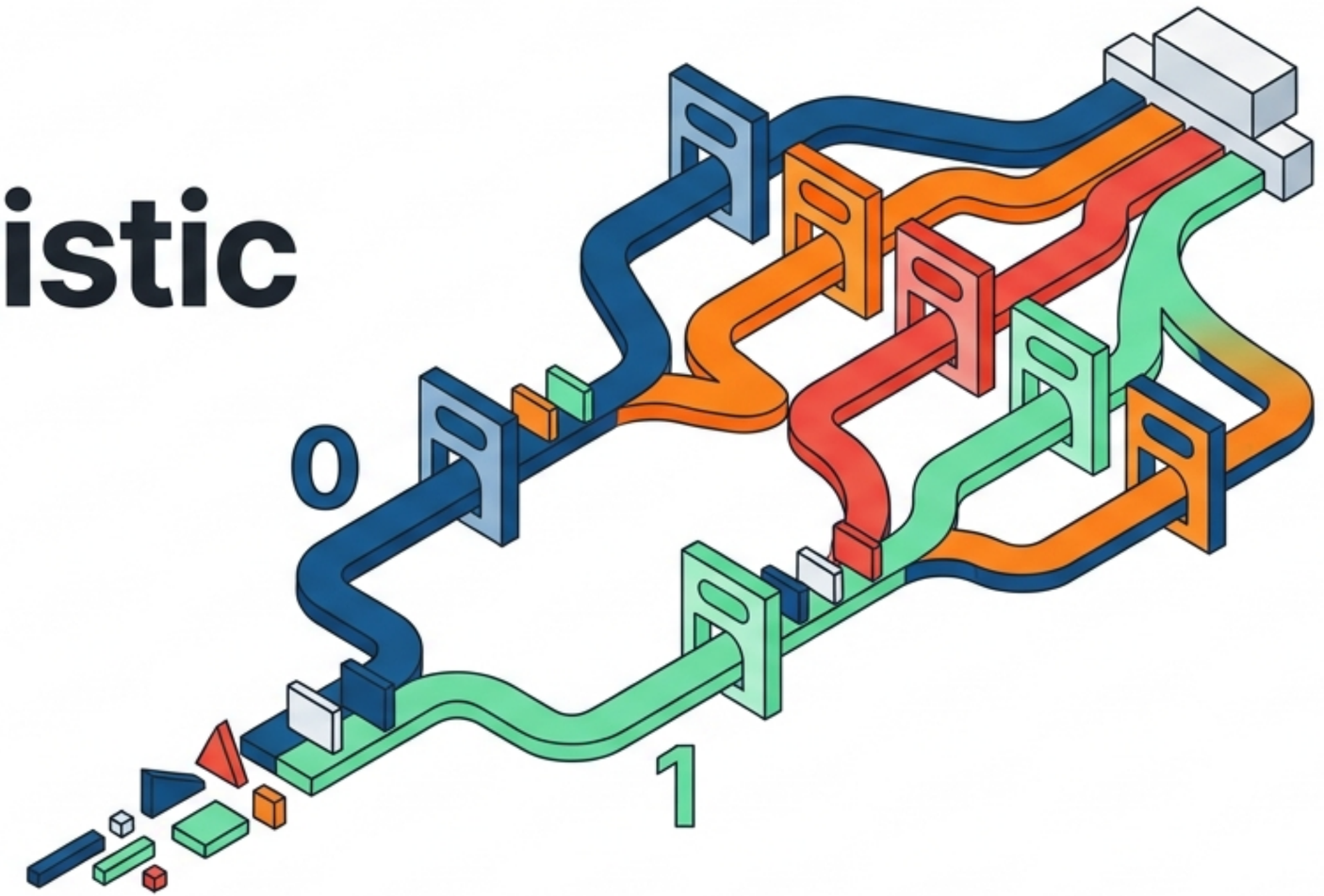
Hình 4-20. Early Stopping (Mũi tên chỉ thời điểm tốt nhất. Các bước sau chỉ làm mô hình tệ hơn...)

Gemini Notebook

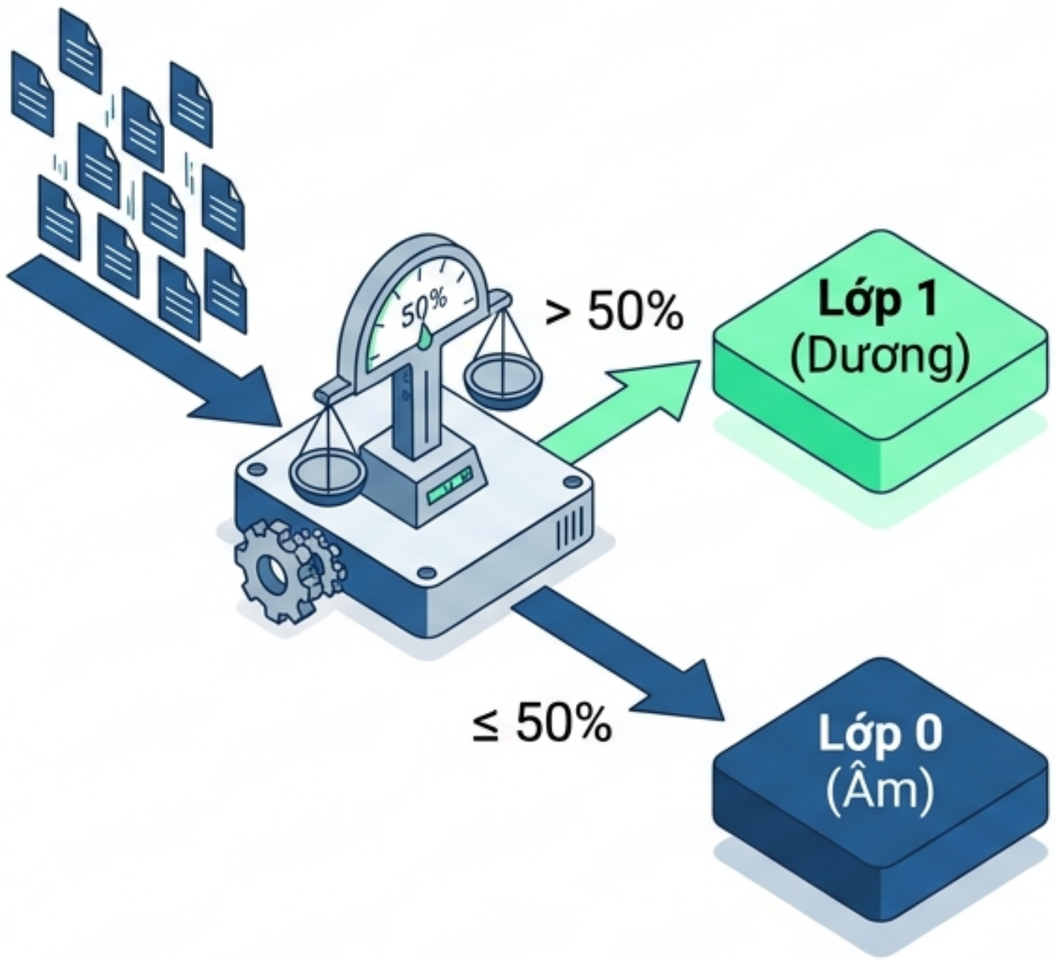
Bài giảng Machine Learning - Chương 4

Hồi quy Logistic & Softmax

Giới thiệu Phân loại Nhị phân và Đa lớp



Gemini Notebook



Bản chất của Hồi quy Logistic

- Dù tên là Hồi quy, nhưng dùng để Phân loại (Nhị phân - 2 lớp).
- Ước tính xác suất thuộc về một lớp (Vd: 70% là thư rác).
- Quy tắc: Xác suất > 50% → Lớp 1; Ngược lại → Lớp 0.

Công thức (9): $\hat{p} = h_\theta(\boldsymbol{x}) = \sigma(\theta^T \boldsymbol{x})$

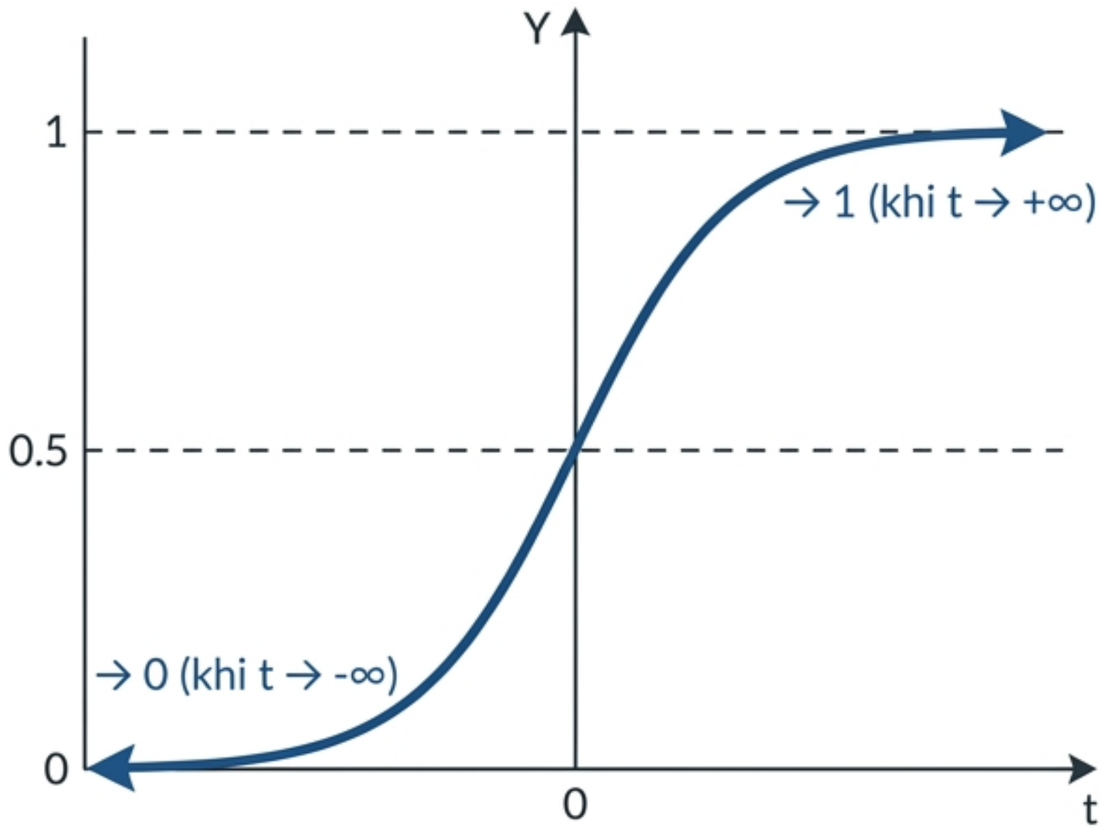
***Ghi chú**: Hoạt động giống Hồi quy tuyến tính (tổng trọng số), nhưng xuất ra hàm logistic.*

Gemini Notebook

Hàm Logistic (Sigmoid)

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

•	**Đầu vào:** Bất kỳ số thực nào (t).
•	**Đầu ra:** Giới hạn trong khoảng (0 → 1).
•	**Xu hướng:** Bất kỳ nhục hình và số nhiớng. t lớn → σ(t) ≈ 1 | t âm lớn → σ(t) ≈ 0.



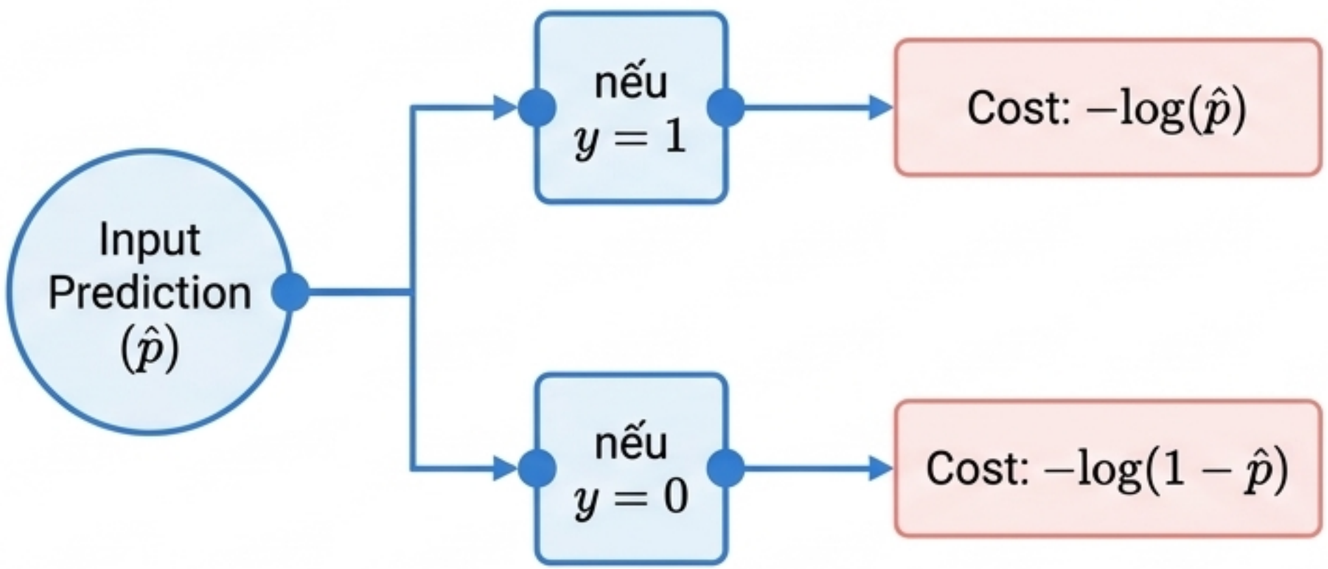
Hình 4-21. Đồ thị Hàm Logistic (Sigmoid)

Gemini Notebook

Hàm chi phí (Cho một trường hợp)

Công thức (10):

$$c(\theta) = -\log(\hat{p}) \quad \text{nếu } y = 1$$
$$c(\theta) = -\log(1 - \hat{p}) \text{ nếu } y = 0$$



Ý nghĩa Toán học: Lỗi sẽ rất lớn (tiến tới vô cùng) nếu mô hình dự đoán xác suất gần 0 cho một lớp thực sự là 1 (và ngược lại). -log(t) rất lớn khi $t \to 0$.

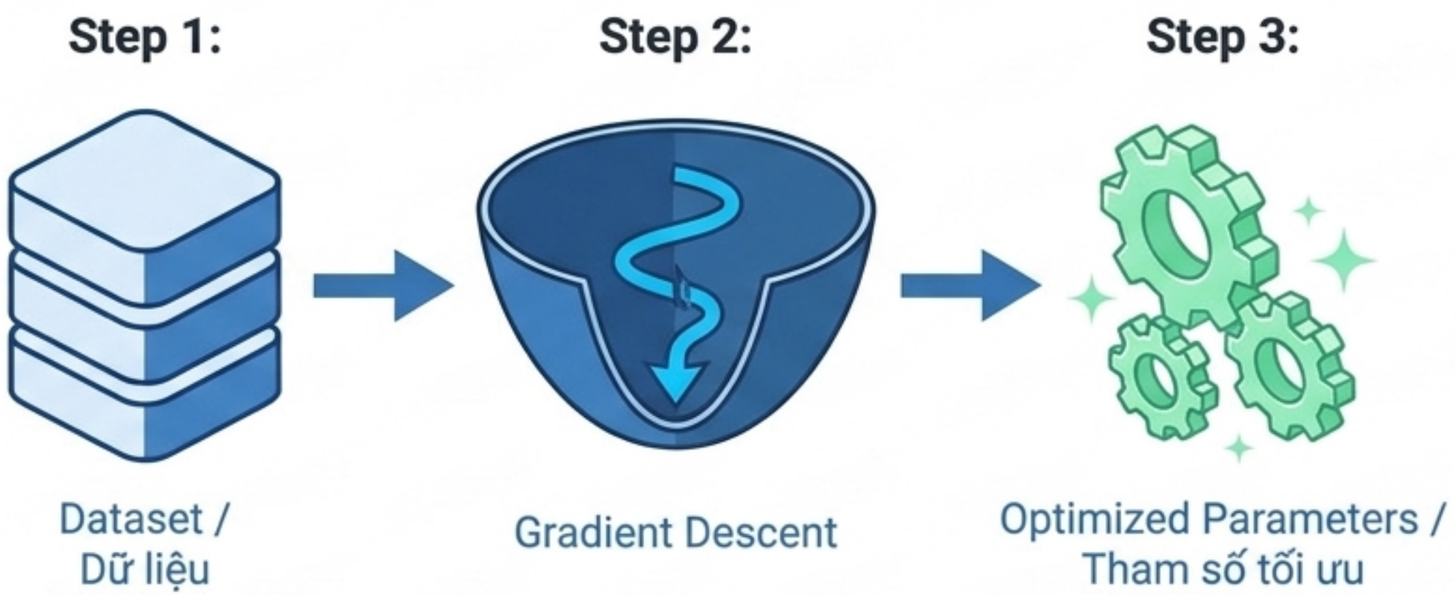
Gemini Notebook

Hàm chi phí **Log Loss** (Toàn tập)

Công thức (11):

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{p}^{(i)}) + (1-y^{(i)})\log(1-\hat{p}^{(i)})\right]$$

- Không có phương trình dạng đóng (không tính trực tiếp được).
- Log Loss là một hàm lồi (Convex function).
- **Giải pháp tối ưu**: Sử dụng thuật toán Gradient Descent.



Gemini Notebook

Minh họa Thực tế: Tập dữ liệu hoa Diên vĩ

Data Profile

Tập dữ liệu: Iris Dataset (150 bông hoa).

Đặc trưng: Chiều dài & chiều rộng (đài hoa, cánh hoa).

3 Loài: Setosa, Versicolor, Virginica.

Bài toán mẫu: Xác định một bông hoa có phải loài Virginica hay không dựa trên chiều rộng cánh hoa?

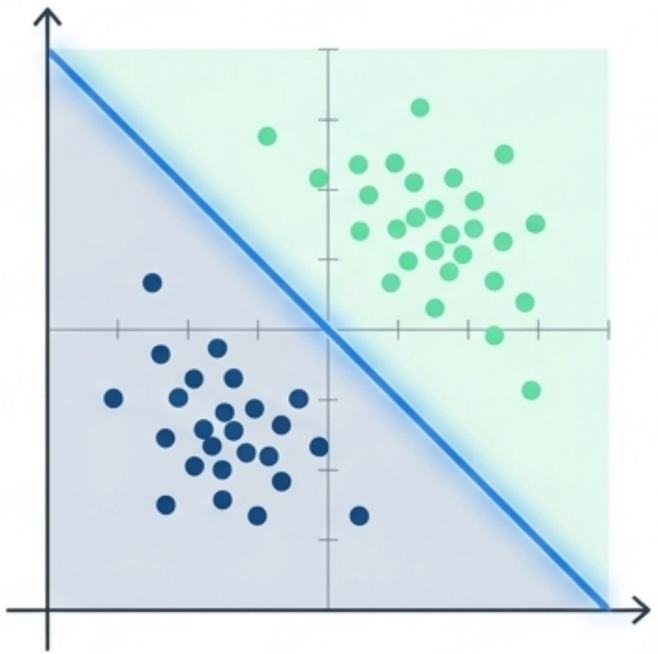
Hình 4-22. Ba loài hoa Diên vĩ.

Gemini Notebook

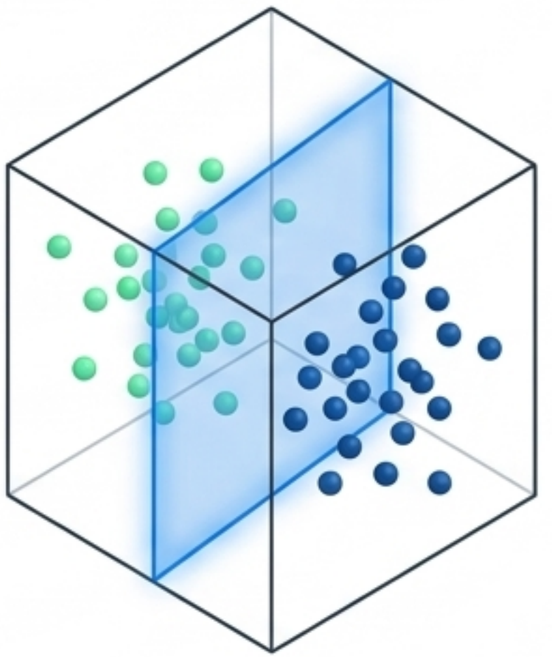
Ranh giới quyết định (Decision Boundaries)

Điểm mô hình thay đổi quyết định (Xác suất = 50%). Hồi quy Logistic luôn có ranh giới tuyến tính.

1 Đặc trưng → Là **1 Điểm** **(Hình 4-23)**



2 Đặc trưng → Là **Đường thẳng** **(Hình 4-24)**



> 2 Đặc trưng → Là **Siêu mặt phẳng**

Gemini Notebook

Hồi quy Softmax (Bước 1: Tính điểm)

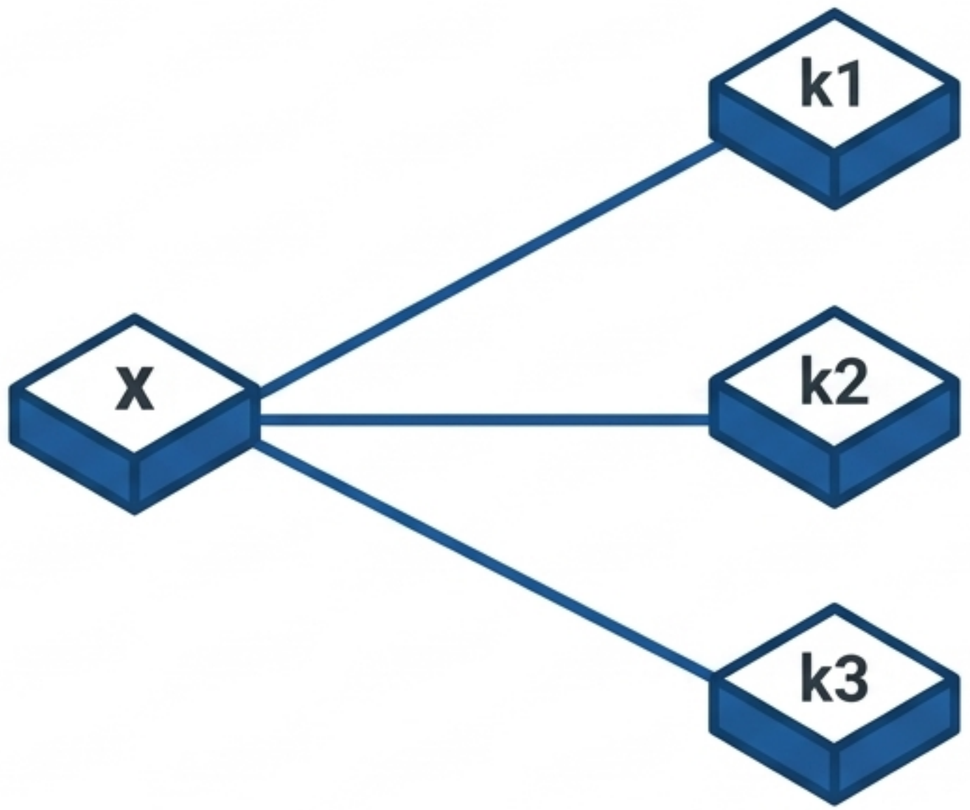
Mở rộng thành Hồi quy Logistic Đa thức.

- Hỗ trợ trực tiếp đa lớp.
- Không cần kết hợp nhiều bộ phân loại nhị phân (OvR).

(Bước 1)

Tính điểm số $s_k(\mathbf{x})$ cho từng lớp k:

$$s_k(\mathbf{x}) = \left(\theta^{(k)}\right)^T \mathbf{x}$$



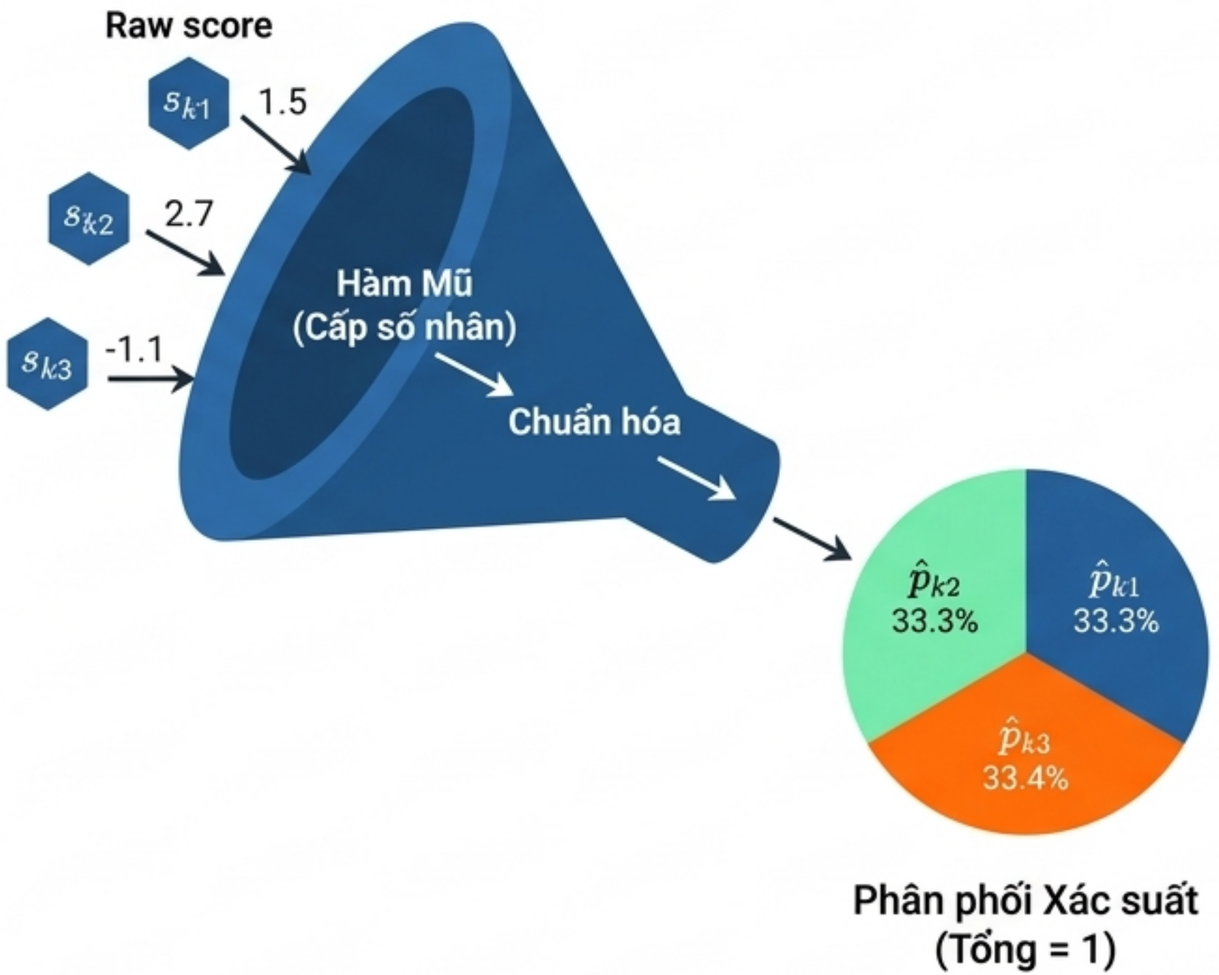
Gemini Notebook

Hàm Softmax (Bước 2: Biến đổi Xác suất)

- Tính cấp số nhân (hàm mũ) của mọi điểm số.
- Chuẩn hóa chúng thành xác suất ước tính ($\hat{p}_k$).

Công thức (12):

$$\hat{p}_k = \sigma(s(x))_k = \frac{\exp\big(s_k(x)\big)}{\sum \exp\big(s_j(x)\big)}.$$



Gemini Notebook

Dự đoán & Hàm chi phí Cross Entropy

- Mô hình dự đoán lớp có xác suất cao nhất.
- Cross Entropy phạt nặng mô hình khi ước lượng xác suất thấp cho lớp đúng (mục tiêu).

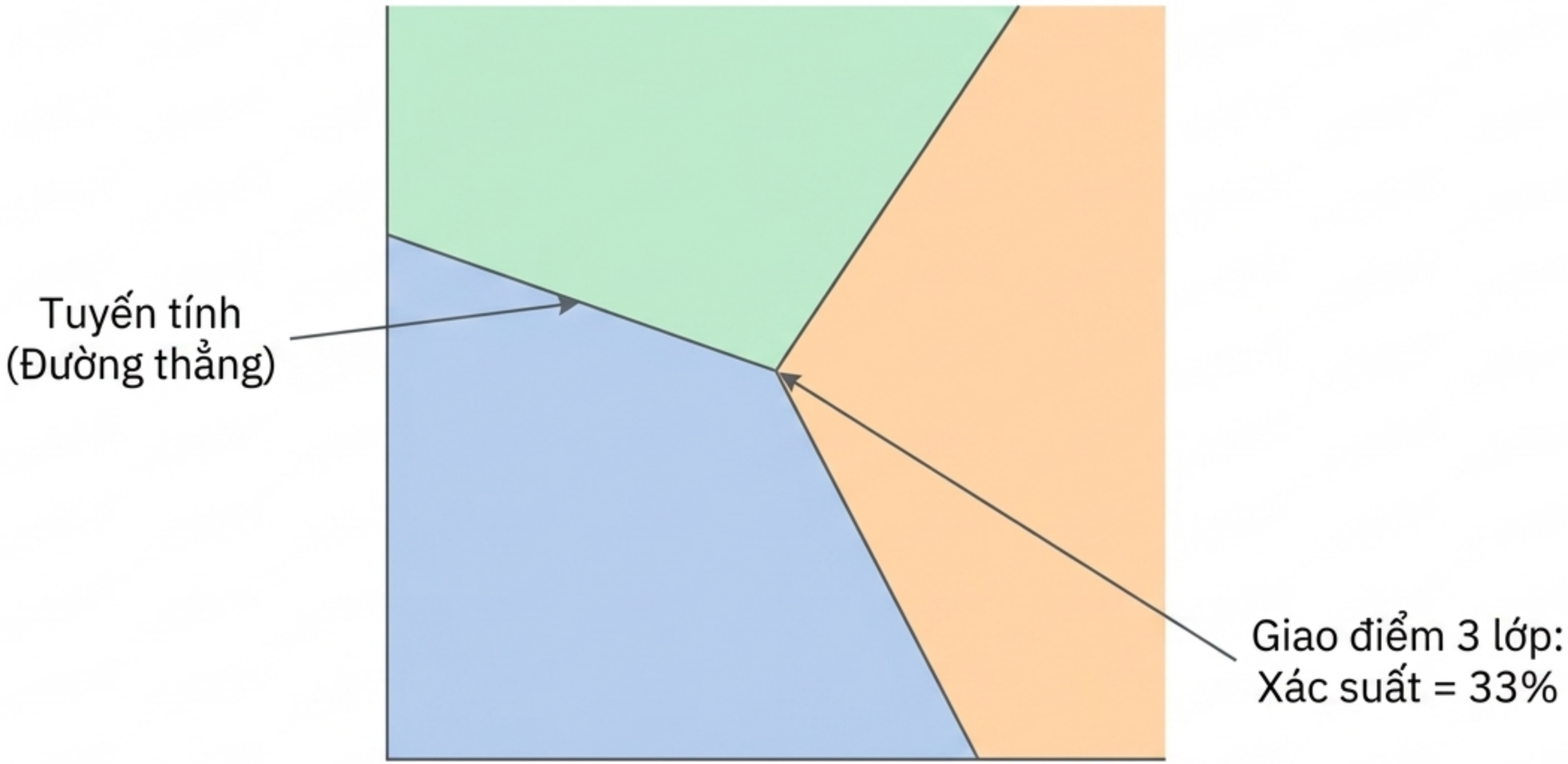
Công thức (13):

$$J(\Theta) = -\frac{1}{m}\sum_{k} y_k^{(i)} \log(\hat{p}_k^{(i)})$$

Mối liên hệ: Khi $K = 2$ (2 lớp), Cross Entropy tương đương chính xác với Log Loss.

Gemini Notebook

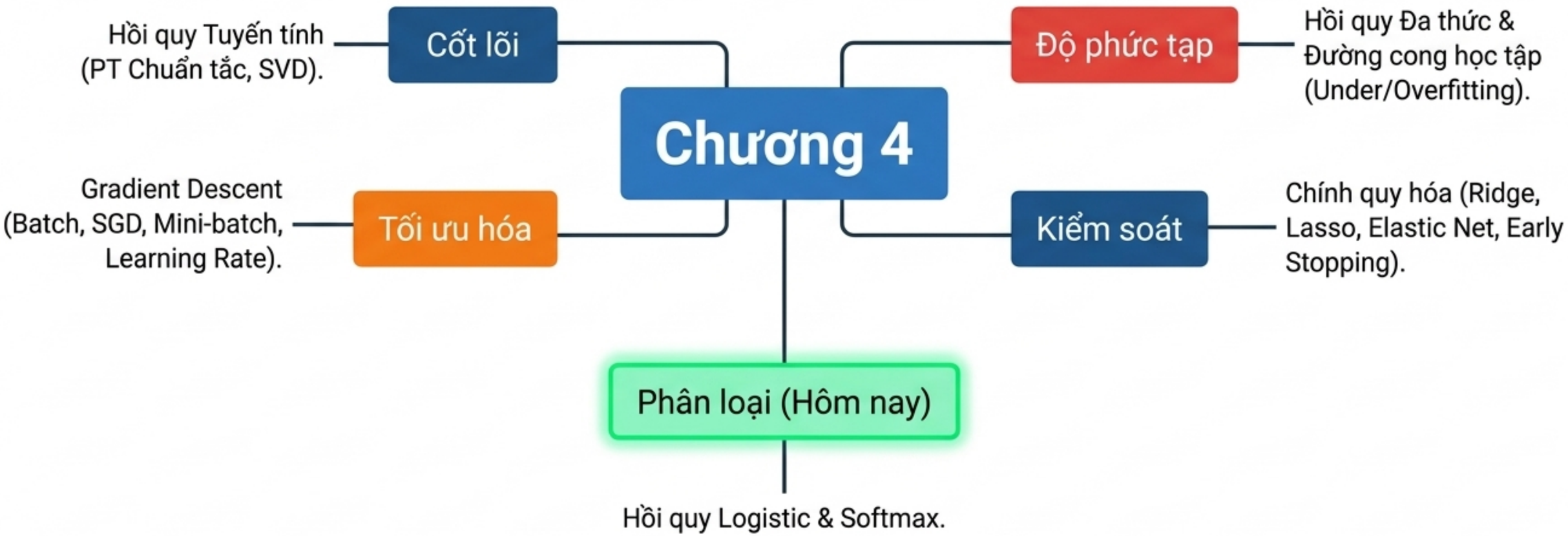
Ranh giới quyết định của Hồi quy Softmax



Hình 4-25. Ranh giới quyết định hồi quy Softmax (Mô phỏng).

Gemini Notebook

Tổng kết Chương 4: Bản đồ Tư duy



Gemini Notebook